



Final Year Project Dissertation
Department of Computer Science
– Project Proposal 2024/2025

Section – A

Students No: 1221344, 1221131 ,1220989

Students Names: Omar Batran, Baha Al-Khateeb, Owais Malash

Course: COMP4200

Email: albatranomar@gmail.com, babaalkhateeb@gmail.com,
owais.malash.p@gmail.com

Section – B

Title of Project: Maskan

Project No:

Supervisor: Dr. Basem Sayrafi.

Key Areas:

Section – C

Student Signature: _____ **Date Submitted:** _____

First Supervisor Name: _____

First Supervisor Signature: _____

Date Approved: ____ / ____ / ____

Section – D

Abstract

The search for rental apartments is becoming more and more difficult due to high demand, scattered listings, and problems in communication between tenants and landlords. Many renters have issues finding apartments that match their requirements, while landlords miss potential tenants offering in the wrong places. These problems reflect badly on tenants, with higher costs, wasted time, and less suitable options.

As a solution to this problem, we want to develop a mobile application that can connect renters with available apartments in a convenient and effective process. Tenants will create an online profile and search for their preferences while landlords are listing their properties with required information, such as location, pricing, amenities, and images.

When a tenant finds an appropriate apartment, they can review the landlord's information and apartment's reviews. After they decide on a rental, they can contact the landlord directly inside the app. After the landlord and tenant reach an agreement, the landlord can mark property as rented, hiding it from other users until it is available again.

To make it more convenient for tenants, our app provides a searching functionality that allows users to filter through the apartments and shows the highest matching and cheapest options it could find. Our system also helps renters during their long searches with a personalized recommendations algorithm that takes into account the user's recent searches and rentals. Additionally, our platform helps landlords with price estimation by comparing the property to similar entries in the market.

Through this system, we aspire to create a more organized and efficient way to reach the rental market, helping both tenants and landlords and saving them money and time.

Table of Contents

Abstract	1
1. Introduction	1
1.1. Aim and Objectives	1
Aims	1
Objectives	1
2. Background	2
2.1. Literature Review	2
2.1.1. Development of Online Based Smart House Renting Web Application	2
2.1.2 Top 3 issues with apartment websites (& how to fix them)	2
2.1.3 Algorithmic Components	3
2.2 Previous Works	6
2.2.1 Booking.com	6
2.2.2 Airbnb	9
2.2.3 Summary	15
2.3 Tools and Technologies	16
2.3.1 React Native	16
2.3.2 PostgreSQL	16
2.3.3 SpringBoot	17
2.3.4 Docker	18
3. System Analysis and Design	19
3.1. Product Description	19
3.1.1. System Objectives	19
3.1.2. System Main Features	19
3.1.3. Operating Environments	19
3.1.4. Constraints	20
3.1.5. Functional Requirements	20
3.1.6. Non-Functional Requirements	24
3.2. Functional Decomposition	25
3.2.1 Actors	25
3.2.2 Use Cases	26
3.2.3. Use Cases Diagram	32
3.3. System Models	33
3.3.1. Class Diagram	33
3.3.2. Sequence Diagram	34
3.3.3. Activity Diagram	36
3.3.4. State Chart Diagram	39
3.4. System Architecture	40
3.4.1. Sub-System	40
3.4.2. Software Architecture	42
3.4.3. Deployment Diagram	43
3.4.4. Component Diagram	44

3.5. Data Management and ERD	45
4. Implementation	46
4.1. Development Environment and Tools	46
4.2. Release-Based Implementation	46
5. Evaluation	48
5.1. Add Listing	48
5.2. Price Estimator	49
5.3. Edit Listing	50
5.4. Registration	51
5.5. Send Booking Request	52
5.6. Send Duplicate Booking Request	53
5.7. Browse Listings By Price Filters	54
5.8. Initiate A Chat with a Tenant	55
5.9. Block A User	56
6. Conclusion and Future works	57
1. Project Summary	57
2. Limitations	57
3. Future Work	57
References	59

Table of Figures

Figure 1-1: Similarity function.	4
Figure 1-2: Clear images of the selected apartment by the user	6
Figure 1-3: Shows the most important features and services for the selected apartment	6
Figure 1-4: Brief description of the apartment.	7
Figure 1-5: Shows the location of the selected apartment	7
Figure 1-6: Area information	7
Figure 1-7: House categories.	8
Figure 1-8: Search filters.	8
Figure 1-9: Wishlists.	9
Figure 1-10: Choose category.	9
Figure 1-11: Choose type	10
Figure 1-12: Choose location	10
Figure 1-13: Apartment features.	11
Figure 1-14: Apartment features	11
Figure 1-15: Apartment photos	12
Figure 1-16: Apartment title	12
Figure 1-17: Apartment description	13
Figure 1-18: Features summary	14
Figure 3-1: Use Case diagram	31
Figure 3-2: Class diagram	32
Figure 3-3: Software Architecture	41
Figure 3-4: Deployment Diagram	42
Figure 3-5: Component Diagram	43
Figure 3-6: ER Diagram	44

List of Tables

Table 1-1: Example of movie recommendation system.	4
Table 3-1: Actors' table	24
Table 4-1: Next Semester plan	46

1. Introduction

Finding an apartment as a family or student is quietly difficult, since it's hard to find an apartment with the desired features, like price, location, and facilities. On the other hand, as a landlord you need to find a way to make your departments reach the most suitable customers for the departments.

Maskan App is an application that aims to build a bridge between the customers and landlords in a seamless and efficient way. Our app lores about two major aspects:

- Landlords: landlords must sign in in our app and provide detailed information about themselves and their departments. Doing so helps our app to demonstrate the department with its features in the best way possible.
- Tenants: families or students can browse the available departments (sorted by the best departments according to the tenant requirements) and take some information about them. To proceed to other functionalities like chatting and recommendation systems the tenant must sign in in our application.

Finally, our Maskan application aims to link the landlord with the department's tenants in a seamless and efficient way. Making it easy for both the landlord and tenants to reach their goals.

1.1. Aim and Objectives

Aims

The aim is to design and develop an application that simplifies housing discovery for residents while providing tools for shared housing management. Within a single, user-friendly application, the product will concentrate on providing a smooth experience for users to search accommodation, and landlords to list properties.

Objectives

1. Enable landlords to list properties and receive automated rental price estimates using a regression model.
2. Implement a dynamic search tool that suggests housing alternatives beyond strict filters.
3. Allow tenants to leave verified reviews for properties to guide future renters.
4. Ensure regional accessibility for seamless use across devices and locations.
5. Simplify apartment discovering with intuitive filters and AI-driven suggestions.

2. Background

2.1. Literature Review

2.1.1. Development of Online Based Smart House Renting Web Application

The house rental market encounters many obstacles, including an inefficient process for locating and renting properties, poor communication between tenants and property owners, and issues of trust concerning property listings. Conventional rental practices often require manual searches, which can be both time-consuming and unreliable. Furthermore, property owners may find it challenging to verify prospective tenants, while tenants struggle to evaluate the credibility of landlords. These complications render the rental process more intricate and less efficient, especially in densely populated urban areas where housing demand is significant.

The study discusses a web-based smart house renting application to address these problems by introducing modern technology solutions into the rental process. The system provides a user-friendly platform where tenants can search for available houses, contact house owners directly, and securely store their identification information. Property owners can verify tenant histories, ensuring that only authenticated tenants contact them. The application fosters trust through the manual verification of property owner details, resulting in a more transparent and dependable rental experience. By utilizing online communication and digital verification, the platform reduces the inefficiencies associated with traditional rental methods.

The research shows how this intelligent rental platform enhances convenience and efficiency within the housing sector. It promotes improved communication, expedites the rental process, and builds trust between tenants and landlords. Additionally, the centralized system streamlines the process of rental listings and property management without needing outside apps. Nevertheless, challenges such as data security and the necessity for widespread adoption persist. Despite these issues, the study highlights the potential of such smart systems to revolutionize the rental market and establish a more accessible and trustworthy housing ecosystem.

While this study introduces solutions for many problems, it leaves out some features that can really help both tenants and landlords. For example, a feature can be added to help the landlords with price estimates or for tenants to compare different prices with the features provided. And while it provides search functionality, it works more like a filter option instead of trying to find different properties that can be better for the user even if they don't exactly fit. This can be very important since the users don't always know what features they want exactly but are more concerned about price.

2.1.2 Top 3 issues with apartment websites (& how to fix them)

The design and functionality of apartment websites is the key to attracting potential renters and enhancing user experience and engagement. One primary challenge faced by such websites is the lack of engaging, floor plan-specific content [1]. This issue/problem causes most of the time a generic user experience, which can fail to get the attention of prospective tenants who are looking for detailed information about a specific apartment or room for their needs. To address this, creating content that highlights unique features of the apartment makes it easier for the users to think of themselves living there [1]. Additionally, the imperfect mobile version of the service, which can make it hard for users attempting to browse on their mobile [1]. Ensuring the mobile experience is fully functional and seamless is very important.

IRJET's research on a web-based flatmate searching system puts the light on the challenges of developing effective web-based housing solutions [2]. While the main idea is based on them, roommate matching is the most important thing rather than apartments and their location or price. The study underlines the need of incorporating advanced search features and assuring the platform's security and reliability, especially in a world increasingly reliant on web-based services [2].

2.1.3 Algorithmic Components

2.1.3.1 Cost Estimation Algorithm

Linear Regression is a fundamental supervised learning algorithm used for predicting numerical values based on input features. Since our app requires cost estimation, and we assume a **linear relationship** between features (e.g., house size, number of rooms, and location score) and cost, Linear Regression is an appropriate choice [3].

Linear Regression is effective when there is a **direct proportionality** between input variables and output [x]. It helps in **interpretable cost estimation**, where each feature contributes linearly to the final cost, making it easier to analyze and understand [4].

The basic equation for **Multiple Linear Regression** is:

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Where:

- Y is the predicted cost.
- x_i are the independent variables (e.g., house size, location score)
- β_0 is the intercept.
- β_i coefficients (weights)
- ϵ error term

This equation represents a **best-fit line** that minimizes the sum of squared residuals (Least Squares Method) [5].

We will train the model using Gradient Descent or the Normal Equation to optimize coefficients [5]. We evaluate model performance using:

- Mean Squared Error (MSE): Measures average squared differences between actual and predicted costs.
- R^2 Score: Explains the proportion of variance captured by the model [6].
- A higher R^2 score (closer to 1) means the model fits the data well.

2.1.3.2 Recommendation Algorithm

The collaborative filtering algorithm is one of the most successful approaches to recommendation generation. It makes use of the user's behavior [7]. The underlying principle is that users who have shown similar preferences or behaviors in the past are likely to have similar preferences in the future [8]. Collaborative filtering can be further divided into user-based, item-based, and model-based approaches [9].

- **Content-Based filtering:** Recommends apartments based on the similarity of their features to the preferences a user has shown in the past [10]. By analyzing attributes of apartments, such as location, price range, and number of bedrooms, and matching these

with a user's profile built from their previous interactions, like viewed listings or saved searches [11].

- **Item-Based filtering:** Recommends apartments which are similar to those a user has previously shown interest in [9]. Users are likely to be interested in other apartments with comparable features or those that are regularly viewed by users who also loved the original flats if they have liked a certain apartment.

A simple example of the movie recommendation system will help us in explaining:

Users	Movie 1	Movie 2	Movie 3	Movie 4
User 1	5	4		5
User 2	4		3	
User 3		1		2
User 4	1	2		

Table 1-1: Example of movie recommendation system.

In this type of scenario, we can see that User 1 and User 2 give nearly similar ratings to the movie, so we can conclude that Movie 3 is also going to be averagely liked by User 1 but Movie 4 will be a good recommendation to User 2, like this we can also see that there are users who have different choices like User 1 and User 3 are opposite to each other. One can see that User 3 and User 4 have a common interest in the movie, on that basis we can say that Movie 4 is also going to be disliked by User 4. This is Collaborative Filtering, we recommend to users the items which are liked by users of similar interest domains [12].

We can also use the cosine similarity between the users to find out the users with similar interests, larger cosine implies that there is a smaller angle between two users, hence they have similar interests. We can apply the cosine distance between two users in the utility matrix, and we can also give the zero value to all the unfilled columns to make calculation easy, if we get smaller cosine then there will be a larger distance between the users, and if the cosine is larger than we have a small angle between the users, and we can recommend them similar things.

$$Similarity = \cos(\theta) = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n B_i^2}}$$

Figure 1-1: Similarity function.

2.1.3.3 Search Result Ranking

After getting the results of the initial search based on the location and filters, we need to rank the results depending on their relevance to the user. There are many different ways to achieve that, mainly:

- **Weighted Scoring:** Commonly used in multi-criteria decision-making (MCDM) to aggregate multiple attributes into a single score. [13]
- **Machine Learning based models:** Techniques like Learning-to-Rank (LTR) leverage historical user interactions for personalized ranking. [14]
- **Hybrid approaches:** Combining heuristic-based scoring with ML-driven optimization for dynamic ranking. [13]

While Machine Learning solutions are adaptable, in our case, the metrics are relatively simple numeric or categorical values. As a result, Heuristic methods, such as weighted scoring, work perfectly fine and are more practical since they are easier to implement with limited data and don't need expensive training.

$$Score = w_0 m_0 + w_1 m_1 + w_3 m_3 + \dots + w_n m_n$$

Where w is the weight of the metrics, m is the normalized score of the metric, and n is the number of metrics used. It must also be noted that the sum of all weights must equal one.

So, in our case, we can use an equation similar to the following:

$$Score = 0.3d + 0.3p + 0.2r + 0.2f$$

Where d is the distance score, p is the price score, r is the rating score, and f is the features score.

Since the values we are using have very different ranges (e.g. rating will be between 1 and 5 stars while the price score can be in the hundreds) we need to normalize the values to a uniform scale. For our case, min-max scaling will work, giving us the values in a 0-1 range [15]. So, we put each value in the following equation:

$$X' = \frac{X - \min(X)}{\max(X) - \min(X)}$$

Where X' is the normalized value, X is the original value, and $\min(x)$ and $\max(x)$ are the minimum and maximum values retrieved from the database.

Finally, the following equations describe how to calculate the score value for each metric:

$$Distance\ Score = 1 - \frac{d - \min(d)}{\max(d) - \min(d)}, \text{ where } d \text{ is the distance}$$

$$Price\ Score = 1 - \frac{p - \min(p)}{\max(p) - \min(p)}, \text{ where } p \text{ is the price}$$

$$Rating\ Score = \frac{r - \min(r)}{\max(r) - \min(r)}, \text{ where } r \text{ is the average users' rating}$$

$$Feature\ Score = \frac{Number\ of\ Matched\ Features}{Number\ of\ Requested\ Features}$$

2.2 Previous Works

2.2.1 Booking.com

Booking.com, founded in 1996 in Amsterdam, has grown into a leading global digital travel company. As part of Booking Holdings Inc., its mission is to simplify travel by leveraging technology to connect millions of travelers with accommodations, transportation, and experiences. It supports both travelers and property owners, helping businesses expand their reach.

The main features of this website are:

- **Filters.** Provides clear and convenient filters for the customers to search for their desired apartments
 1. Filters based on the country, city.
 2. Filters based on the numbers of rooms, adults and children number.

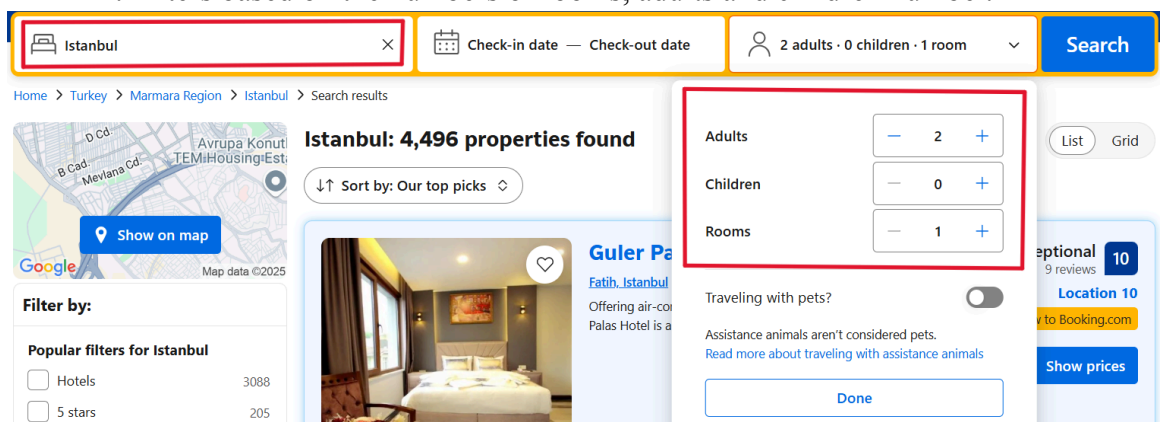


Figure 1-2: Clear images of the selected apartment by the user

Sultann Apart

📍 Cankurtaran Meydanı Sokak No:3 3, Fatih, 34122 Istanbul, Turkey – [Show map](#) – Train Access



Figure 1-3: Shows the most important features and services for the selected apartment

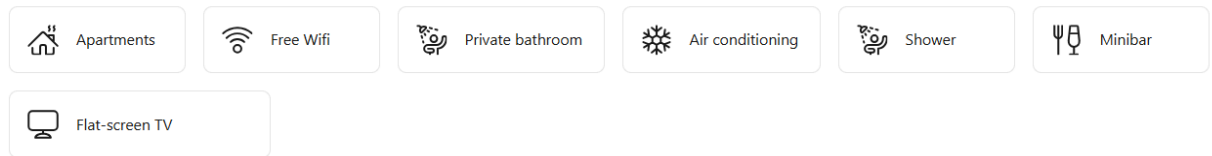


Figure 1-4: Brief description of the apartment.



Figure 1-5: Shows the location of the selected apartment

Area info [See availability](#)

[Show map](#)

Located in the historic old town ×
Discover Istanbul's old town, which boasts more history than almost any other place in the world. Explore nearby icons like the **Hagia Sophia** and the **Grand Bazaar**.

What's nearby Cankurtaran Meydan Parkı 50 m Odabaşı Parkı 350 m Great Palace Mosaic Museum 500 m Tavşanlı Sahil 550 m German Fountain 600 m Basilica Cistern 600 m Çayiroğlu Parkı 700 m Hippodrome of Constantinople 700 m Obelisk of Theodosius 800 m Serpent Column 850 m	Top attractions Topkapi Palace 900 m Istanbul Archaeological Museum 900 m Column of Constantine 1.3 km Galata Tower 3 km Taksim Square 4.7 km Dolmabahce Clock Tower 5 km Dolmabahce Palace 5 km Maiden's Towers 8 km 15 July Martyrs Bridge 9 km Istanbul Sapphire 11 km	Public transit Train - Cankurtaran 50 m Train - Sultanahmet Tram Station 800 m Subway - Vezneciler 2.5 km Subway - Haliç 3 km Closest Airports Istanbul Sabiha Gokcen International Airport 39 km Istanbul Airport 40 km
Restaurants & cafes Restaurant - Meydan Börek 0 m Restaurant - Otel Ipekyolu Restaurant 30 m Restaurant - Korecan Restaurant 30 m	Natural Beauty Forest - Banaz Parkı 9 km	

Figure 1-6: Area information

Provides nearby stores, restaurants and top facilities and attractions of the area.

Booking.com attracts a diverse range of travellers, from solo adventurers and families to business professionals and groups seeking unique experiences. Whether planning a weekend getaway, a long vacation, or a business trip, users looking for a wide variety of accommodations, seamless booking, and reliable customer support will find Booking.com to be the ideal platform.

2.2.2 Airbnb

Airbnb was born in 2007 when two hosts welcomed three guests to their San Francisco home, and has since grown to over 5 million hosts who have welcomed over 2 billion guest arrivals in almost every country across the globe. Every day, hosts offer unique stays and experiences that make it possible for guests to connect with communities in a more authentic way.

The main features of this website are:

1. Traveler Side

- **Wide Variety of Listings** – Stay in apartments, houses, castles, treehouses, houseboats, or even igloos.

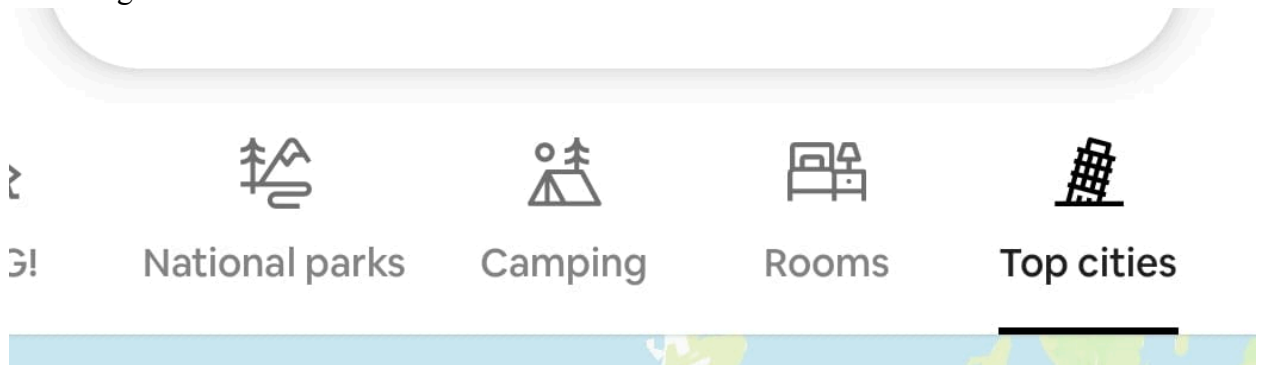
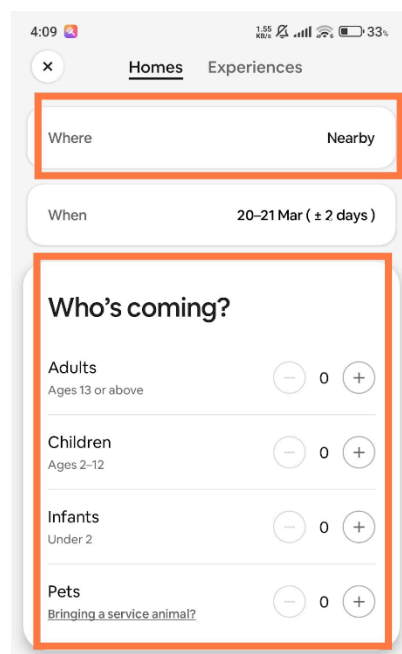


Figure 1-7: House categories.

- **Filters for Personalized Search** – Search by location, type of stay and number of people.



- **Wishlists** – Save favorite properties and plan trips easily.

Wishlists

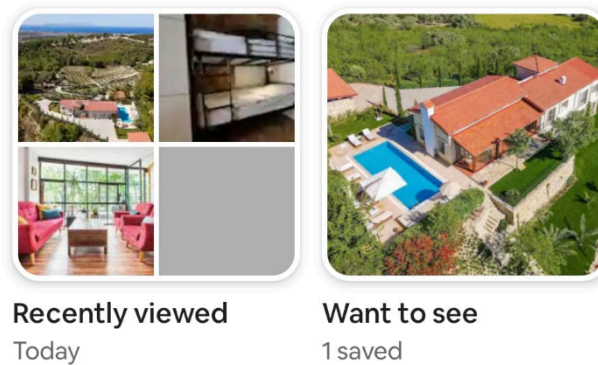


Figure 1-9: Wishlists.

- **Superhost & Verified Listings** – Look for highly rated hosts and verified properties for a better experience.
2. Host Side
 - **Easy Listing Process** – Upload photos, set prices, and list a property within minutes.
 - **Host Dashboard & Insights** – Track bookings, revenue, and guest reviews easily.
 - **Automated Messaging & Booking Management** – Communicate with guests and manage reservations seamlessly.
 3. The process of listing:
 1. Describing the place

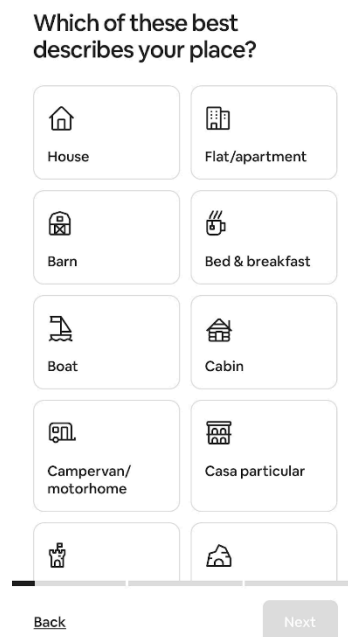


Figure 1-10: Choose category.

2. Type of the place for the guests

What type of place will guests have?

An entire place
Guests have the whole place to themselves.

A room
Guests have their own room in a home, plus access to shared spaces.

A shared room in a hostel
Guests sleep in a shared room in a professionally managed hostel with staff on-site 24/7.

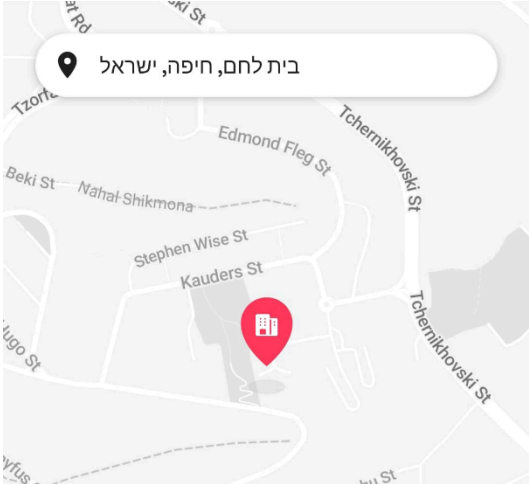
Figure 1-11: Choose type

3. The location of the place

Where's your place located?

Your address is only shared with guests after they've made a reservation.

בית לחם, חיפה, ישראל



Where's your place located?

Your address is only shared with guests after they've made a reservation.

Enter your address

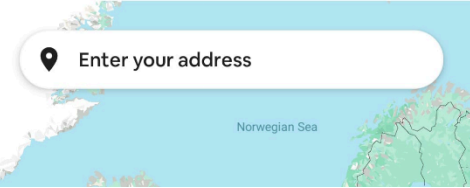


Figure 1-12: Choose location

4. Basic information about the apartment

Share some basics about your place

You'll add more details later, such as bed types.

Guests	−	4	+
Bedrooms	−	1	+
Beds	−	1	+
Bathrooms	−	1	+

Figure 1-13: Apartment features.

5. Select features of the apartment

Tell guests what your place has to offer

You can add more amenities after you publish your listing.

What about these guest favourites?

 Wifi	 TV
 Kitchen	 Washing machine
 Free parking on premises	 Paid parking on premises
 Air conditioning	 Dedicated workspace

[Back](#)[Next](#)

Figure 1-14: Apartment features

6. Uploading photos of the apartment

Add some photos of your house

You'll need 5 photos to get started. You can add more or make changes later.

+ Add photos

📷 Take new photos

Figure 1-15: Apartment photos

7. A title for the house

Now, let's give your house a title

Short titles work best. Have fun with it – you can always change it later.

great house with good views

5 characters available

Figure 1-16: Apartment title

8. More description of the house

Next, let's describe your house

Choose up to 2 highlights. We'll use these to get your description started.

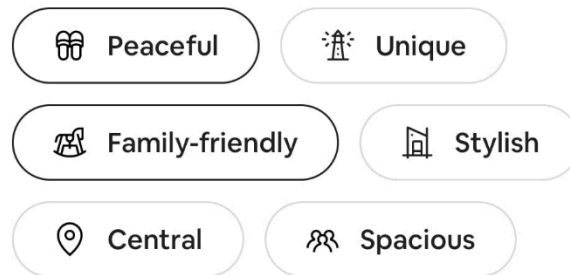


Figure 1-17: Apartment description

Airbnb is designed for a diverse audience, including travellers looking for unique and affordable accommodations, hosts who want to earn money by renting out their properties, and businesses or remote workers seeking flexible stays. It caters to budget travellers, luxury tenants, families, solo adventurers, and digital nomads. Additionally, Airbnb Experiences attract tourists interested in immersive local activities, while long-term rental options appeal to students and professionals needing extended stays. Whether for leisure, work, or adventure, Airbnb provides a wide range of options to suit different needs.

2.2.3 Summary

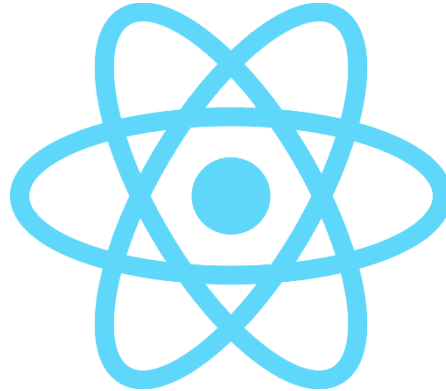
Feature	Booking.com	Airbnb	Our App
Types of Stays	Multiple (Hotels, apartments, hostels, resorts, B&Bs, guesthouses, vacation homes)	Multiple (Homes, apartments, cabins, tiny houses, treehouses, houseboats, yurts, castles)	Multiple (University Residents, Family, Villa ..)
Filters & Search	Extensive filters: price range, amenities, accessibility, sustainability, breakfast included	Advanced filters: pet-friendly, Superhost, flexible check-in, Wi-Fi speed, unique stays	Advanced filters: price range, location, resident features, nearest facilities)
Price comparison	Can compare multiple hotel prices in the same area	No direct price comparison, but dynamic pricing for listings	Can compare multiple hotel prices in the same area
Location on Google Map	Yes	Yes	Yes
Cost Prediction for Hosts	No	No	Yes
Automated messaging	No	Yes	Yes
Clear details of the apartment	Yes	Yes	Yes
Wishlist	Yes	Yes	Yes
Ranking System	Yes	Yes	Yes

Figure 1-18: Features summary

2.3 Tools and Technologies

2.3.1 React Native

React Native is a popular open-source framework for building cross-platform mobile applications using JavaScript and React. It allows developers to create apps for both iOS and Android using a single code-base while running with near native performance. It also brings the React programming paradigm from the web to mobile development.



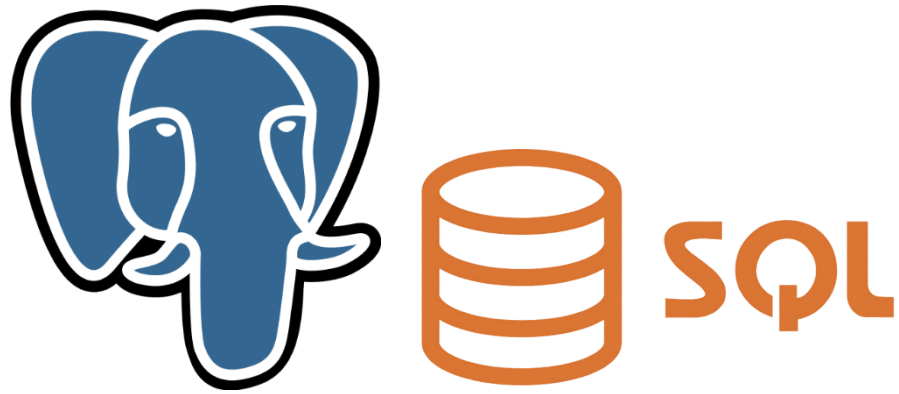
JavaScript, one of the most popular programming languages worldwide, is the main language used in React Native. The syntax extension JSX (JavaScript XML), which enables developers to write HTML-like code within JavaScript, is used by React Native. Complex interface management is made simpler by this combination, which makes it possible to create reusable and dynamic UI components.



We choose React Native because our app needs to run on both major operating systems. Additionally, using React Native, we can reuse parts of the code for building a website using React, saving both time and effort in writing the frontend.

2.3.2 PostgreSQL

PostgreSQL is a highly capable open-source relational database management system recognized for its strength, flexibility, and adherence to SQL standards. It offers modern features, including ACID compliance, the ability to handle complex queries, and support for JSON, which makes it suitable for various applications. With its focus on high performance, scalability, and data integrity, PostgreSQL is used in small and large applications alike.



For our project, PostgreSQL's handling of both structured and semi-structured data allows for all the data we may need. It also has powerful modern features (e.g. Geo Queries) that will help in the complex functionalities in our application (e.g. search functionality).

2.3.3 SpringBoot

Spring Boot is a Java-based framework designed to simplify the development of stand-alone, production-grade applications. It provides a robust backend architecture for handling business logic, RESTful APIs, and database interactions.



The primary programming language used in Spring Boot is Java, a widely-used, object-oriented programming language known for its portability, scalability, and extensive ecosystem of libraries and frameworks.

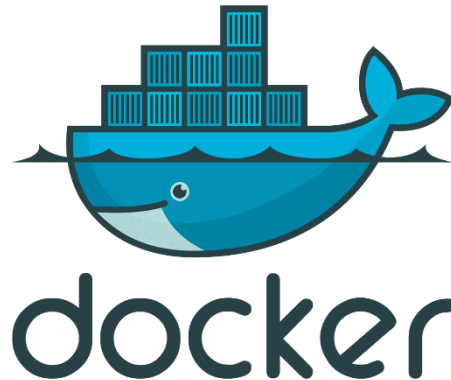


We chose Spring Boot because it simplifies backend development through its auto-configuration feature, which reduces boilerplate code and speeds up the development process. It includes embedded servers, dependency management, metrics, externalized configuration, and health

checks, which streamline the setup and monitoring of backend services. With extensive documentation, Spring Boot makes it easier to find solutions to common problems and learn best practices.

2.3.4 Docker

Docker is a platform that enables developers to package applications into containers and executable components that combine application source code with the operating system libraries and dependencies required to run that code.



We selected Docker because it eliminates problems caused by environment mismatches, like "it works on my machine," by encapsulating the program and its dependencies and guaranteeing consistency across development, testing, and production environments.

3. System Analysis and Design

3.1. Product Description

Maskan is a mobile application designed to connect landlords and apartment tenants, such as families and students, in a seamless and efficient way. Landlords can sign up to list detailed information about their apartments, helping showcase their properties effectively. Tenants can browse and filter available apartments based on their preferences like price, location, and facilities. To access features like chatting and personalized recommendations, tenants must create an account. Maskan aims to simplify the rental process for both landlords and tenants, making it easier for each to achieve their goals.

3.1.1. System Objectives

- Enable landlords to list properties and receive automated rental price estimates using a regression model.
- Implement a dynamic search tool that suggests housing alternatives beyond strict filters.
- Allow tenants to leave verified reviews for properties to guide future renters.
- Ensure regional accessibility for seamless use across devices and locations.
- Simplify apartment discovering with intuitive filters and AI-driven suggestions.

3.1.2. System Main Features

- Secure user authentication and profile management, including registration, login, logout, and profile editing.
- Browsing, searching, and viewing available house listings with detailed property information.
- Managing favorite properties by saving and organizing preferred listings.
- Facilitating communication between users and landlords through inquiries, booking requests, and real-time chat.
- Receiving notifications related to bookings, messages, and important updates.
- Providing a rating and review system for houses and landlords after rental experiences.
- Enabling landlords to manage house listings by adding, editing, or removing properties.
- Offering landlords a dashboard to track listing performance and view statistics.
- Supporting rent price suggestions.

3.1.3. Operating Environments

Makan will be developed to support both mobile and backend services efficiently across various platforms. The mobile app is built using **React Native**, enabling cross-platform compatibility for **iOS** and **Android** devices with a single codebase. The backend system, developed with **Spring Boot** (Java), handles business logic and RESTful API services, while **PostgreSQL** is used for robust, scalable database management. Together, these technologies ensure high performance, portability, and the ability to support complex data operations such as search functionalities.

To maintain consistency across development, testing, and production environments, the entire system is containerized using **Docker**.

3.1.4. Constraints

To use the mobile application, an active internet connection is required. Additionally, users must provide a valid email address in order to create an account and access our features. The ability to search or operate on the app is restricted to registered users only, ensuring that all activities within the system are tracked and managed securely.

3.1.5. Functional Requirements

UR.1 The system shall allow potential users to register into the system.

SR.1.1 The system shall provide a registration form where users must fill and submit to register to the system:

- **Full Name:** String, max 50 characters, cannot be empty.
- **Email Address:** String, max 100 characters, must be a valid email format.
- **Password:** String, min 8, max 64 characters.
- **Password Confirmation:** String, min 8, max 64 characters.
- **Phone Number:** String, exactly 10 digits, must be numeric.

SR.1.2 The system shall validate that all mandatory fields are completed before allowing registration.

SR.1.3 The system shall enforce strong password rules (minimum 8 characters, mix of uppercase, lowercase, numbers, and special characters).

SR.1.4 The system shall check if the provided email address is already registered and prevent duplicate accounts.

SR.1.5 The system shall send a verification email after registration, requiring users to verify their email.

UR.2 The system shall allow potential users to login into the system.

SR.2.1 The system shall allow tenants to log in using their registered email and password:

- **Email Address:** String, max 100 characters, must match registered email.
- **Password:** String, max 64 characters, validated against hashed password.

SR.2.2 The system shall lock the user from trying to login into account after 5 consecutive failed login attempts.

SR.2.3 The system shall allow users to reset their password securely if forgotten (via email verification).

SR.2.4 The system shall show appropriate error messages for incorrect login attempts (without revealing which field is wrong for security reasons).

SR.2.5 The system shall provide an option to login using Google Account.

UR.3 The system shall allow users to browse available houses using filters with sort functionality.

SR.3.1 The system shall display a list of available houses retrieved from the database.

SR.3.2 The system shall allow users to filter houses by price range, number of bedrooms, and location.

SR.3.3 The system shall allow users to sort the list of houses by price ascending or descending.

SR.3.4 The system shall allow users to sort the list of houses by newest listings first.

SR.3.5 The system shall allow users to reset all applied filters to default.

- UR.4** The system shall allow users to view the detailed information for each listing.
- SR.4.1** The system shall allow users to click on a listing title or image to access its detailed view page.
 - SR.4.2** The system shall present listing details including title, description, price, availability status, seller information, 10 images of the house/apartment.
 - SR.4.3** The system shall handle missing or incomplete listing data gracefully by displaying appropriate placeholder messages.
 - SR.4.4** The system shall allow users to navigate back to the listings page without losing their original search or filter context.
- UR.5** The system shall allow users to save houses (apartments) to a favorites list (saved list).
- SR.5.1** The system shall provide an option on each house/apartment listing to add it to the user's favorites list.
 - SR.5.2** The system shall allow users to view all houses/apartments they have added to their favorites list.
 - SR.5.3** The system shall allow users to remove houses/apartments from their favorites list.
 - SR.5.4** The system shall ensure that only authenticated users can add, view, or remove favorites.
 - SR.5.5** The system shall display an appropriate error message if saving to favorites fails.
- UR.6** The system shall provide contact information of the landlord of each apartment.
- SR.6.1** The system shall allow users to view the landlord's contact information from the apartment details page.
 - SR.6.2** The system shall restrict access to landlord contact information based if it's a registered user.
 - SR.6.3** The system shall display a message if landlord contact information is missing or unavailable.
- UR.7** The system shall allow users to send inquiries or booking requests to landlords.
- SR.7.1** The system shall provide an option on each property listing page visible to users.
 - SR.7.2** The system shall allow users to enter a custom message (up to 500 characters) when sending an request.
 - SR.7.3** The system shall make tenants enter a detailed request to the landlord (number of tenants, relationship of tenants, gender, willingness to share rooms).
 - SR.7.4** The system shall allow users to choose a preferred move-in date and rental duration as part of a booking request.
 - SR.7.5** The system shall validate that the user is logged in before they can submit an inquiry or booking request.
 - SR.7.6** The system shall notify the landlord via an in-app notification when a user sends a new booking request.
 - SR.7.7** The system shall display a confirmation message to the user after successfully submitting a request.
 - SR.7.8** The system shall prevent duplicate booking requests for the same property by the same user.
 - SR.7.9** The system shall allow users to view the status of their inquiries and booking requests.
- UR.8** The system shall provide a chat or messaging interface to enable tenants to talk with landlords.
- SR.8.1** The system shall allow tenants to initiate a chat session with landlords.
 - SR.8.2** The system shall allow landlords to reply to tenant messages in real time.
 - SR.8.3** The system shall store chat history between tenants and landlords for future reference.
 - SR.8.4** The system shall notify users of new incoming messages.
 - SR.8.5** The system shall allow users to view a list of active and past conversations.

- SR.8.6** The system shall restrict chat access to authenticated users only.
- SR.8.7** The system shall allow users to block other users in the chat.
- SR.8.8** The system shall support sending text messages in the chat interface.
- UR.9** The system shall send notifications for booking updates and messages for both landlords and tenants.
- SR.9.1** The system shall send a mobile push notification to the tenant when a landlord responds to a booking request.
- SR.9.2** The system shall notify both landlord and tenant when a booking status changes (e.g., confirmed, cancelled, or modified).
- UR.10** The system shall allow tenants to view and manage their pending booking requests. This includes enabling them to cancel a booking request before confirmation.
- SR.10.1** The system shall display a list of all pending booking requests for the currently logged-in tenant.
- SR.10.2** The system shall provide an option to cancel any pending booking request.
- SR.10.3** The system shall confirm cancellation actions with the tenant before finalizing the cancellation.
- SR.10.4** The system shall update the status of a cancelled booking request to "Cancelled".
- SR.10.5** The system shall notify the tenant upon successful cancellation of a pending request.
- UR.11** The system shall allow tenants to rate and review houses and landlords after renting.
- SR.11.1** The system shall display a rating and review form to a tenant after the rental period has ended.
- SR.11.2** The system shall allow tenants to submit a numerical rating (1 to 5 stars) for the rented house.
- SR.11.3** The system shall allow tenants to submit a numerical rating (1 to 5 stars) for the landlord.
- SR.11.4** The system shall allow tenants to write a textual review for the rented house.
- SR.11.5** The system shall allow tenants to write a textual review for the landlord.
- UR.12** The system shall allow users to edit their profile information.
- SR.12.1** The system shall have an option to edit profile details.
- SR.12.2** The system shall allow users to update the following fields:
- Full Name: String, max 50 characters cannot be empty.
 - Email Address: String, max 100 characters, must be a valid email format.
 - Phone Number: String, 10–15 digits, must be numeric.
 - Password: String, min 8, max 64 characters.
- SR.12.3** The system shall validate that all fields follow the correct format before allowing the change (Valid phone number, Valid email, and strong password).
- SR.12.4** The system shall require users to enter their current password to confirm changes.
- SR.12.5** The system shall notify the user upon successful profile update or if any error occurs.
- UR.13** The system shall allow users to log out of their account.
- SR.13.1** The system shall provide an option to log out of account.
- SR.13.2** The system shall terminate the user's session and invalidate any associated authentication tokens upon logout.
- SR.13.3** The system shall redirect the user to the home page or login page after a successful logout.
- SR.13.4** The system shall display a message stating "Logged out successfully" upon successful logout.
- SR.13.5** The system shall clear all sensitive data stored on the client side upon logout.

UR.14 The system shall allow landlords to add a new house listing.

SR.14.1 The system shall provide a form where landlords must fill house details, including:

- Title: String, max 100 character, cannot be empty.
- Description: String, max 5000 characters.
- Price: float, positive.
- Full address: String, max 200 characters, cannot be empty.
- Number of bedrooms: int, range 1-20
- Amenities: List for pre-defined options.
- Photos: JPG/PNG, size $\leq$ 5MB each, max number of files: 10.

SR.14.2 The system shall validate that all required fields are complete and valid before submission.

SR.14.3 The system shall notify the landlord upon successful listing creation.

SR.14.4 The system shall automatically mark newly created listings as “Available” by default.

UR.15 The system shall allow landlords to maintain and manage their house listings.

SR.15.1 The system shall display all listings created by the logged-in landlord.

SR.15.2 The system shall allow landlords to update all listing details.

SR.15.3 The system shall allow landlords to upload, delete, or replace listing images.

SR.15.4 The system shall allow landlords to delete a listing.

SR.15.5 The system shall confirm all listing modifications before saving changes.

SR.15.6 The system shall notify the landlord upon successful listing updates or if errors occur.

UR.16 The system shall allow landlords to view a dashboard with listing statistics and booking requests.

SR.16.1 The system shall provide a dashboard view for landlords summarizing key statistics.

SR.16.2 The dashboard shall include metrics such as total listings, active listings, pending booking requests, and total bookings.

SR.16.3 The system shall display recent booking activity and listing views per property.

SR.16.4 The system shall allow landlords to filter booking requests by status.

SR.16.5 The system shall display booking requests along with their current status (Pending, Processing, Accepted, Declined).

UR.17 The system shall allow landlords to manage booking requests.

SR.17.1 The system shall display all booking requests for the landlord’s properties.

SR.17.2 The system shall allow landlords to change the booking request to “Processing” (the process of visiting and negotiation between the tenant and the landlord) or decline each request.

SR.17.3 The system shall prompt the landlord to confirm any action before taking a final decision.

SR.17.4 The system shall notify the tenant of the decision (accepted, declined, or in progress) via app notifications.

SR.17.5 The system shall update the booking status to accordingly.

SR.17.6 The system shall update the booking request’s status to “Accepted” if the booking was accepted.

SR.17.7 The system shall change the listing status to “Booked” after a booking request is accepted.

UR.18 The system shall allow landlords to view ratings and reviews of their houses.

SR.18.1 The system shall allow the landlord to view reviews of their houses.

- SR.18.2** The system shall display reviews with corresponding tenant ratings (1–5 stars) and comments.
- SR.18.3** The system shall show the average rating for each property.
- SR.18.4** The system shall allow landlords to sort reviews by date or rating.
- UR.19** The system shall allow landlords to rate and review the tenants that booked their listings, after the end of their agreement.
- SR.19.1** The system shall display a rating and review form to the landlord after the rental period has ended.
- SR.19.2** The system shall allow landlords to submit a numerical rating (1 to 5 stars) for the tenant.
- SR.19.3** The system shall allow landlords to write a textual review about the tenant.
- SR.19.4** The system shall restrict rating and review submissions to tenants who rented their listings.
- UR.20** The system must provide an Auto Price Estimator to suggest a rent price for new listings.
- SR.20.1** The system shall provide landlords with an option to estimate their properties price.
- SR.20.2** The estimator shall consider factors such as number of rooms, location, and amenities.
- SR.20.3** The system shall display a suggested price range based on the estimator's output.
- SR.20.4** The landlord shall have the option to accept or ignore the suggested price.
- SR.20.5** The system shall continuously update the estimator's model with current market data to improve accuracy.

3.1.6. Non-Functional Requirements

- **Performance**
 - Page loading time must be less than 3 seconds under a 4G network.
 - The system shall generate the Price Estimation within 2 seconds after receiving all input data.
 - The system shall send the notifications within a maximum time of 10 seconds.
- **Security**
 - All sensitive operations must be performed over HTTPS (TLS 1.3).
 - The system shall securely store passwords.
 - Chat messages must use end-to-end encryption with AES 256-bit.
 - Password reset tokens must expire after 15 minutes.
- **Usability**
 - The app must work on devices with screen sizes from 320px to 1920px.
 - Support for English and Arabic languages at launch.
 - Error messages must be user-friendly (e.g., “Invalid Password” instead of technical errors).

3.2. Functional Decomposition

3.2.1 Actors

Actor	Description
User	The user is either a tenant or landlord and can register to the app, log in into the system, and manage their account information.
Tenant	Tenants create an account on the app and browse, search, and view the listings, send booking requests, message landlords, review their rented properties and landlords, and are notified about the important operations that happen to their bookings or accounts.
Landlord	Landlords create an account on the app and manage their own listings, message Tenants, and accept or decline booking requests.
Notification Service	System service responsible for delivering notifications to Users. It shows the app's notifications to the user when a specific event happens.
Google Maps Service	External service integrated to the application to provide location-based functions. It enables users to view and search the desired location with maps.
Price Estimation Service	System algorithm responsible for calculating the estimated price of the landlords' listing

Table 3-1: Actors' table

3.2.2 Use Cases

Use Case Name	Register
Actors	User
Description	Create a user account, entering all needed accounts and personal information.
Preconditions	None
Postconditions	The new user registered and is saved in the system.
Trigger	The user selects the register option.
Main Flow	1. The user selects the register option. 2. The user enters required account and personal information. 3. The system validates the entered information. 4. The system registers and saves the new account. 5. The system shows a registration success message.
Alternative Flows	● A1. The entered information is invalid → System shows error message. ● A2. The email is already used by another user → System shows a message informing the user that email is already used.

Use Case Name	Log In
Actors	User
Description	Users can log in into the system.
Preconditions	The users must be registered in the system
Postconditions	The user have access to the system
Trigger	The user enters their email and password on the login page then clicks the Login button.
Main Flow	1. User navigates to the login page. 2. User enters an email and password. 3. System verifies the data. 4. System grants access to the user. 5. Users can use the system.
Alternative Flows	● A1. “Remember Me” button is selected → System stores Landlords/Tenants data. ● A2. Invalid Email or Password → System displays error message

Use Case Name	Manage Listing
Actors	Landlords
Description	This use case allows users to create, view, update and delete listings.
Preconditions	The Landlords must be logged in.
Postconditions	The listings database is updated based on the landlord's action.
Trigger	The landlord performs an action on a listing (add, edit, delete, view)
Main Flow	1. The landlord navigates to the listings page. 2. The landlord selects action (add new listing or edit/delete existing one). 3. System performs the action and updates the listings. 4. System notified success to the actor.
Alternative Flows	• A1. Search for listings → Landlord filters the listings. • A2. Multiple listings → Landlord selects multiple listings at once. • A3. Invalid inputs while updating or adding → System display error message for the landlord

Use Case Name	Add Listing
Actors	Landlords.
Description	This use case allows the landlord to create a new listing by entering its details.
Preconditions	Landlord must be logged in.
Postconditions	The new listing is saved in the system and becomes available for the tenants.
Trigger	The landlord selects the Add listing option.
Main Flow	1. Landlord logs into the system 2. Landlord clicks "Add listing" 3. System display add listing form 4. Landlord fills out listing details 5. Actor submits the form 6. System validates the data and saves the listing 7. System notifies the landlord (success message).
Alternative Flows	• A1. Save as Draft -> Actor chooses to stop adding and continue later. • A2. Missing Required Fields → System shows error message • A3. Images (file) Upload Error → System shows error message

Use Case Name	Browse Listings
Actors	User
Description	Users can browse through the listings currently on the app.
Preconditions	The user is logged in.
Postconditions	The user views the available listings.
Trigger	The user chooses the browse option.
Main Flow	1. The user chooses the browsing option. 2. The system shows the user the currently available listings.
Alternative Flows	• A1. No listings are available for the system → The system shows a message informing the user.

Use Case Name	Search Listings
Actors	User
Description	Users can use the filtering and smart search to look for listings.
Preconditions	The user is logged in.
Postconditions	The user views the filtered listings.
Trigger	The user selects the search option.
Main Flow	2. The user selects the search option. 3. The user chooses the search filters. 4. The system shows the user the matching results and orders them accordingly.
Alternative Flows	• A1. No listings match the user's search → The system shows a message informing the user that there are no matching results. • A2. The user doesn't select any filters → The system does the normal browsing behaviour.

Use Case Name	View Listing
Actors	User
Description	Users can view listing details.
Preconditions	• The user is logged in. • The listing is available to view.

Postconditions	The user views the listing details.
Trigger	The user chooses the view listing option.
Main Flow	1. The user chooses the view listing option. 2. The system shows the user the listing details. 3. The user chooses an action to do (view landlord profile, request booking, etc)
Alternative Flows	• A1. The user exists without choosing an option → The system goes back to its previous state.

Use Case Name	Add Listing to Favourites
Actors	Tenant
Description	This case allows a tenant to save a listing to their favorites list.
Preconditions	• The user is logged in. • Listing must be available.
Postconditions	The desired listing is saved to the tenant's favourites list.
Trigger	The user clicks the add to Favorites option on a listing.
Main Flow	1. Tenant logs into the system 2. Tenant browses the available listings. 3. Tenant clicks add to Favorites option 4. System saves the listing to the tenant's favorites list 5. System confirms the action
Alternative Flows	• A1. Remove from favorites list → user clicks the same option on the listing to remove the listing from the list. • A2. Tenant not logged in → Tenant must be logged in to add to the list.

Use Case Name	Booking a Listing Request
Actors	Tenant
Description	This use case allows a tenant to view listings and send booking requests to the owner of the listing.
Preconditions	• Tenants must be logged in. • Must be at least one available listing.
Postconditions	• Booking requests are submitted and stored in the system. • Notification is sent to the listing owner

Trigger	The tenant views a listing and clicks the “Request Book” button.
Main Flow	1. Tenant views listings 2. Tenant selects a listing and clicks “Request Book” button 3. Tenant enters request details 4. System validate the data and save the request 5. System notifies the listing owner of the request 6. System shows confirmation message to the tenant
Alternative Flows	● A1. Booking unavailable listing → System shows error message ● A2. Missing Required Fields → System shows error message

Use Case Name	Manage a Booking Request
Actors	Landlord
Description	The landlord either accepts or declines a booking request from a tenant.
Preconditions	The landlord is logged in to the account the request is directed to.
Postconditions	The booking’s status and property status is updated.
Trigger	The landlord selects a booking request.
Main Flow	1. The landlord selects the view booking request option. 2. The landlord selects the accept/decline option. 3. The system shows a message confirming the action. 4. The system notifies the tenant of the landlord’s decision.
Alternative Flows	● A1. The landlord goes back without selecting an option → The system takes him/her back to the requests page.

Use Case Name	Review property
Actors	Tenants
Description	The tenants can leave reviews for properties they rented and their landlords.
Preconditions	● The tenant is logged in. ● The tenant rented the property he/she wants to review.
Postconditions	The review is saved to the system.
Trigger	The tenant chooses the review option.

Main Flow	1. The tenant chooses the review option. 2. The tenant chooses a rating out of 5. 3. The tenant enters the review text. 4. The tenant chooses the save option.
Alternative Flows	• A1. The tenant saves the review without a text -> The system saves the review • A2. The tenant doesn't choose a rating -> The system shows an error message to the user and asks him/her to choose a rating.

3.2.3. Use Cases Diagram

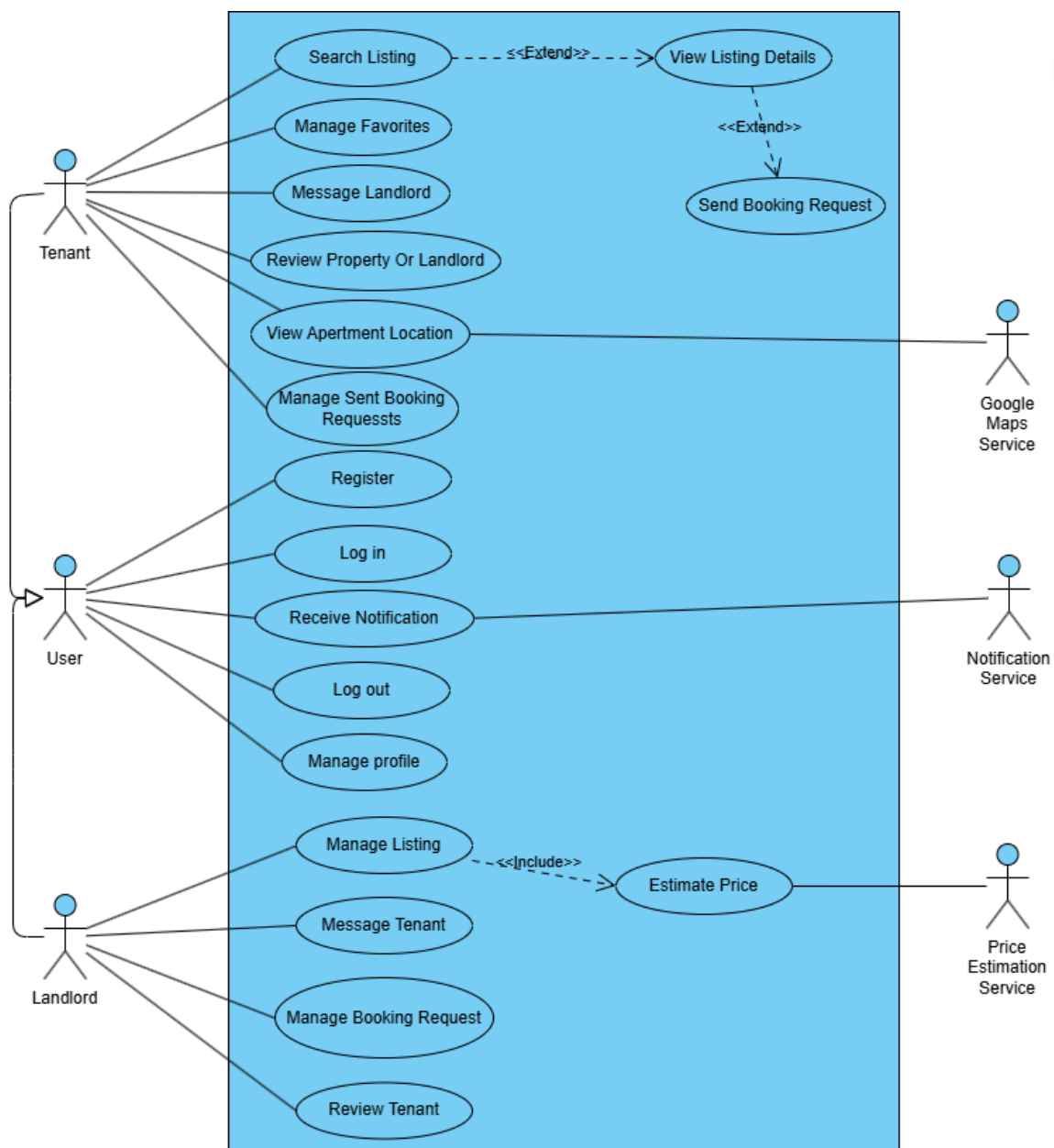


Figure 3-1: Use Case diagram

3.3. System Models

3.3.1. Class Diagram

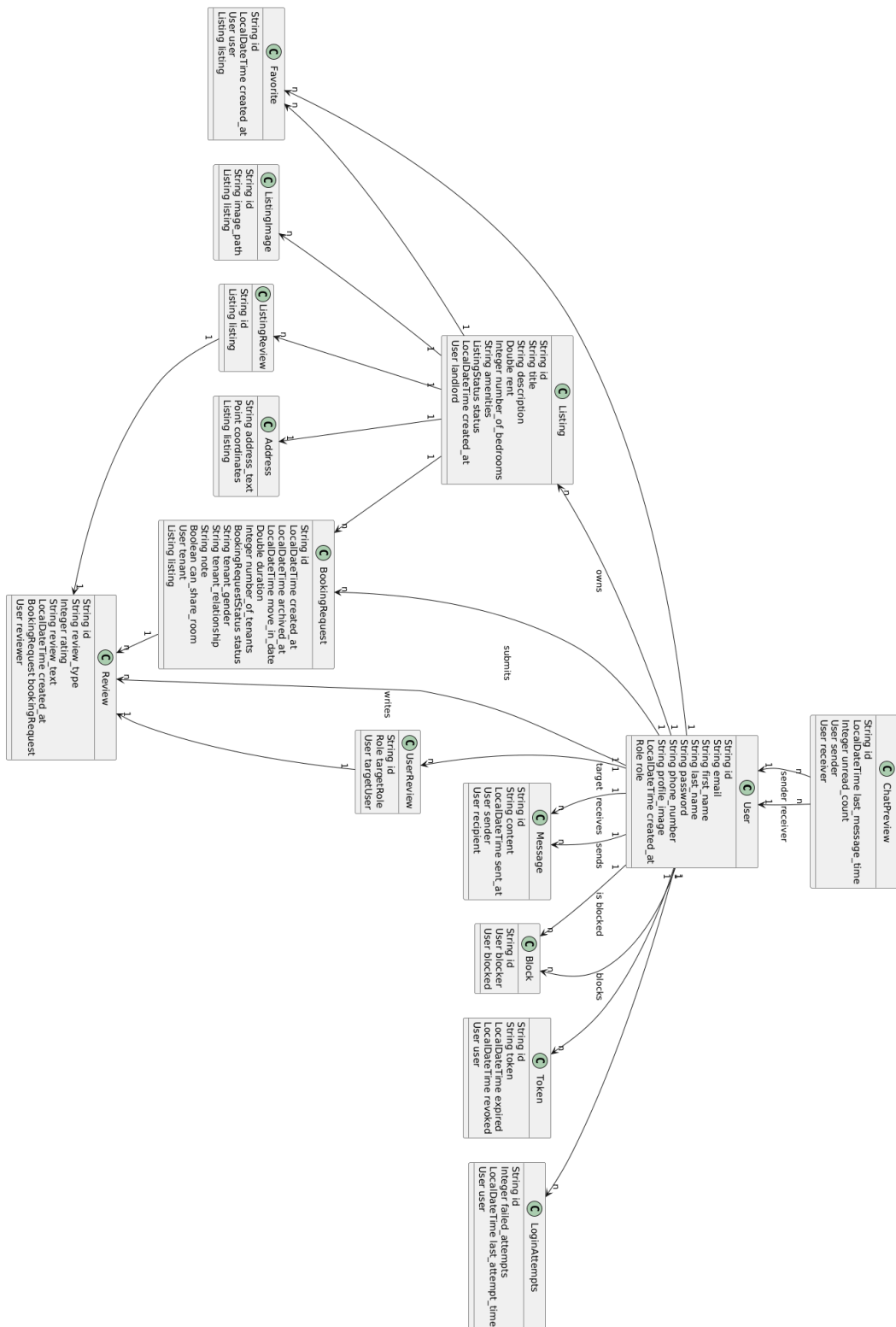
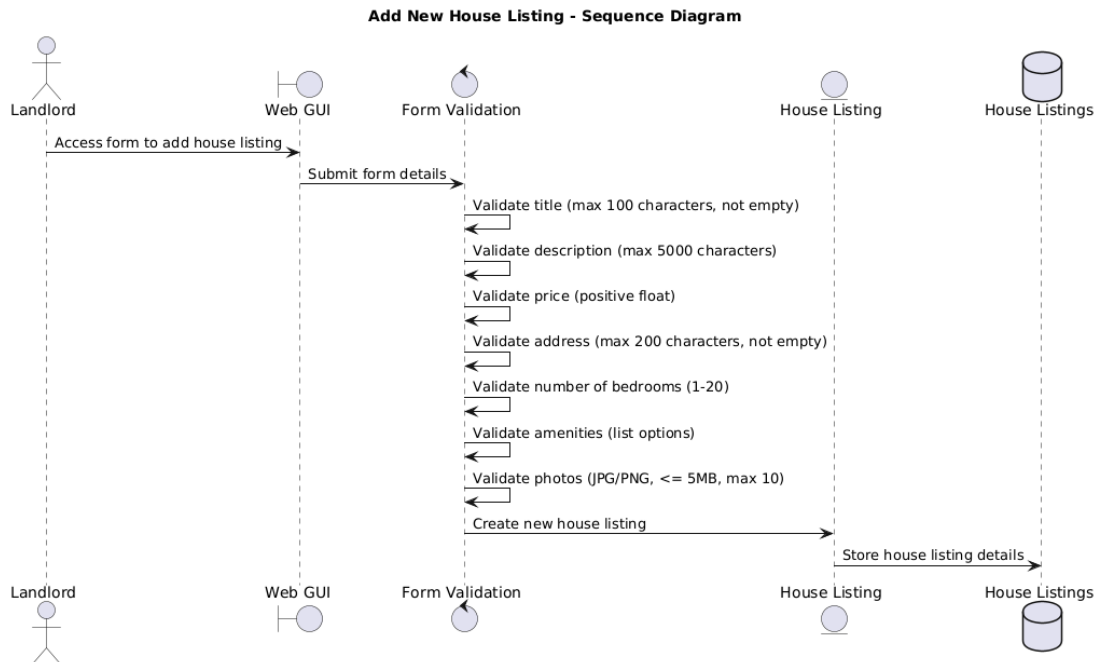


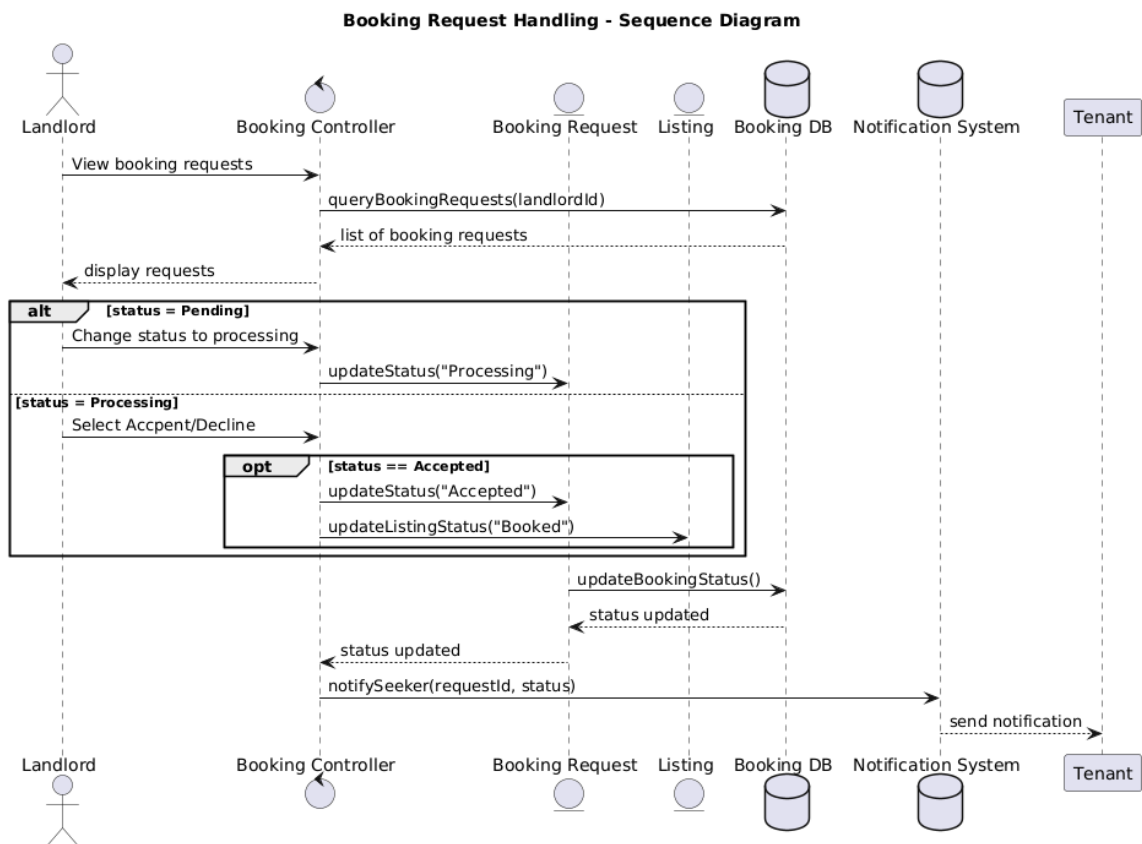
Figure 3-2: Class diagram

3.3.2. Sequence Diagram

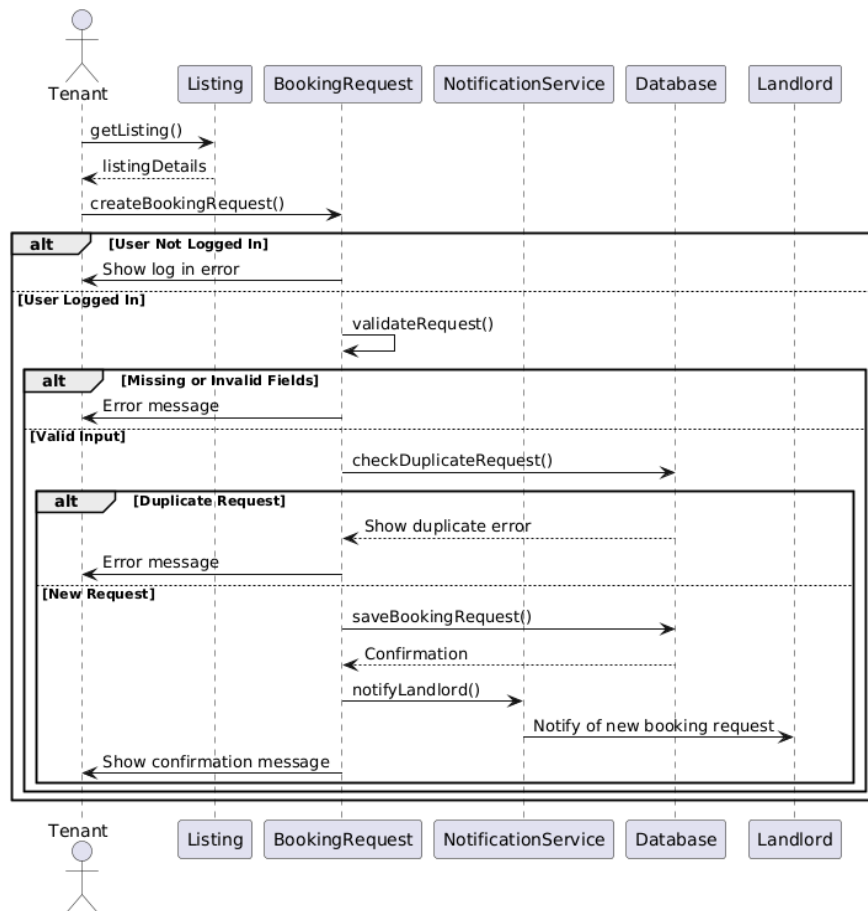
3.3.2.1 Adding a Listing



3.3.2.2 Manage Booking Request

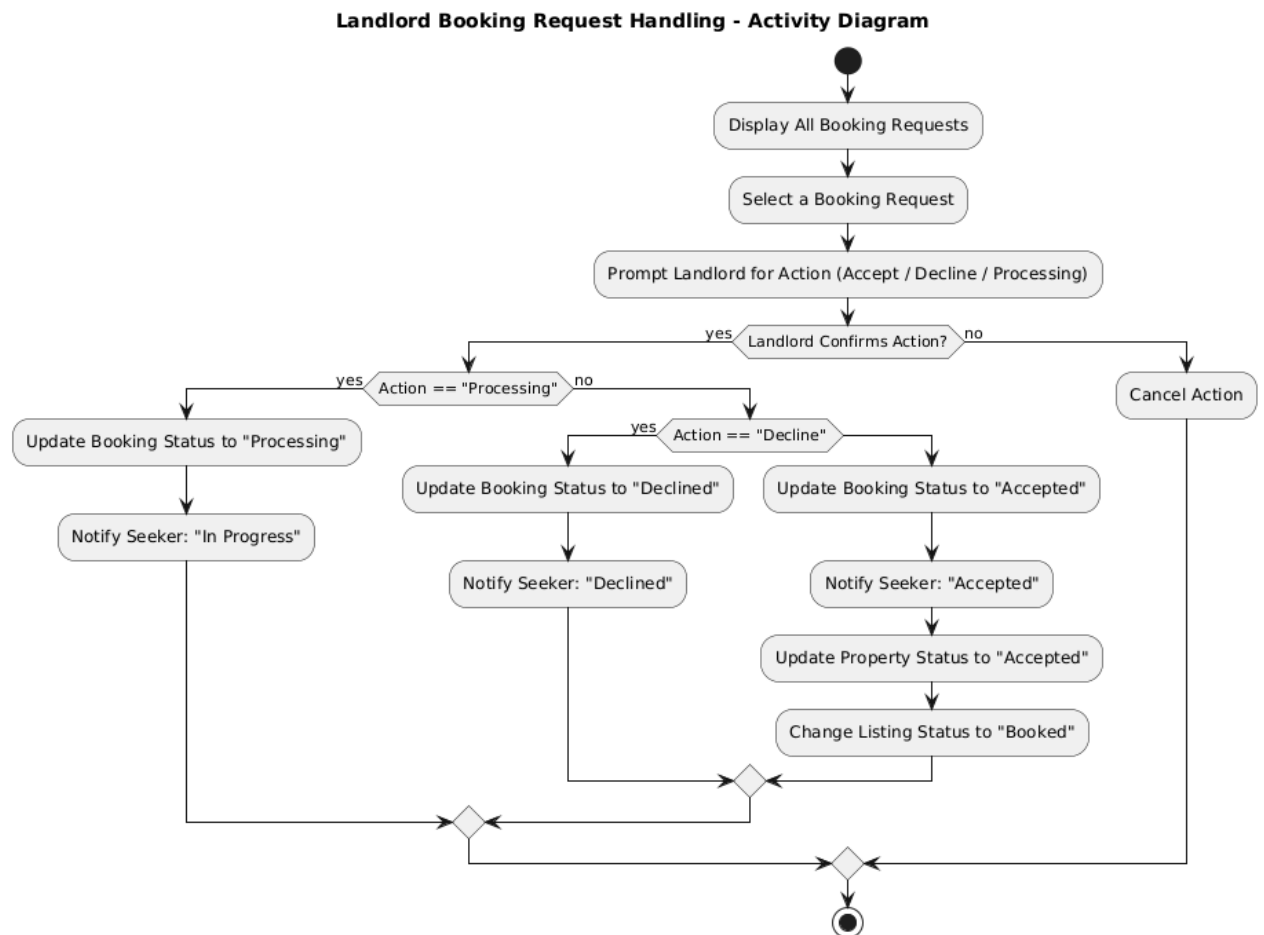


3.3.2.3 Make Booking Request

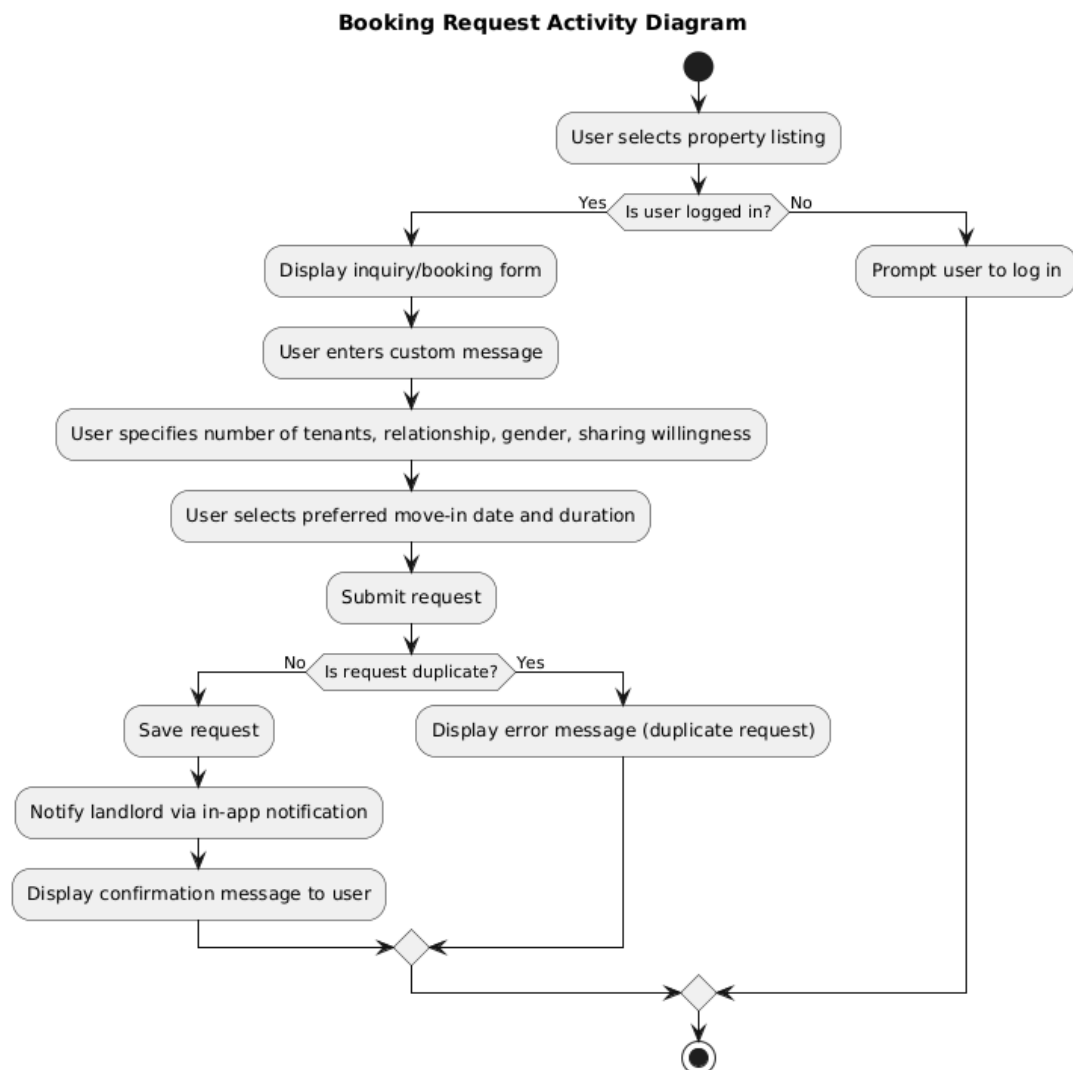


3.3.3. Activity Diagram

3.3.3.1 Manage Booking Request

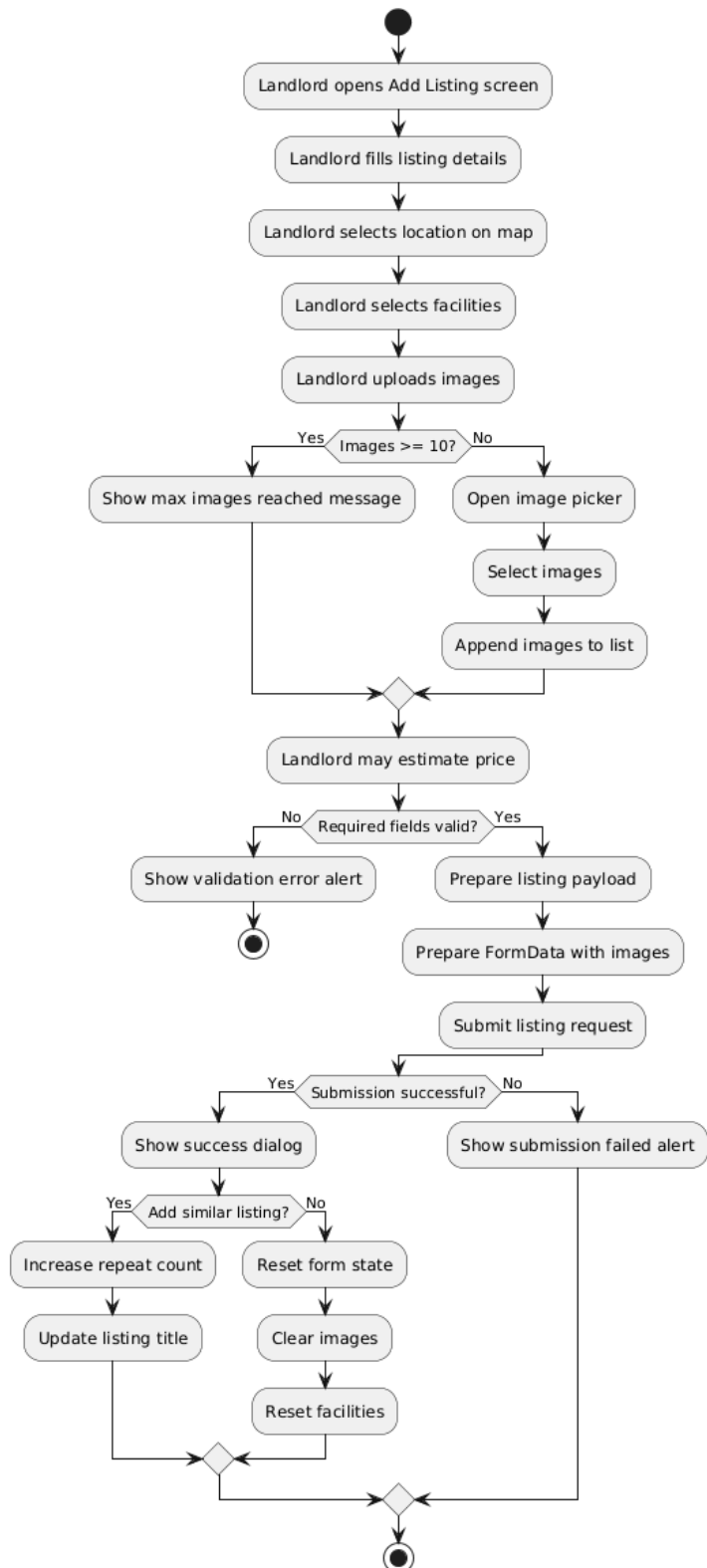


3.3.3.2 Booking request activity diagram



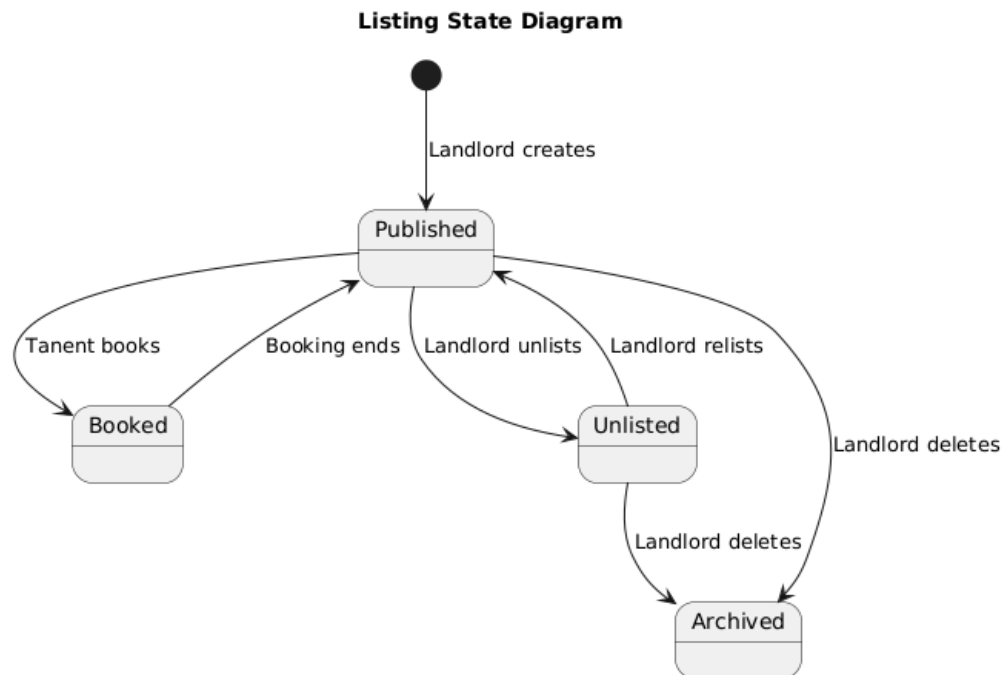
3.3.3.3 Adding a Listing

Add Listing Activity Diagram

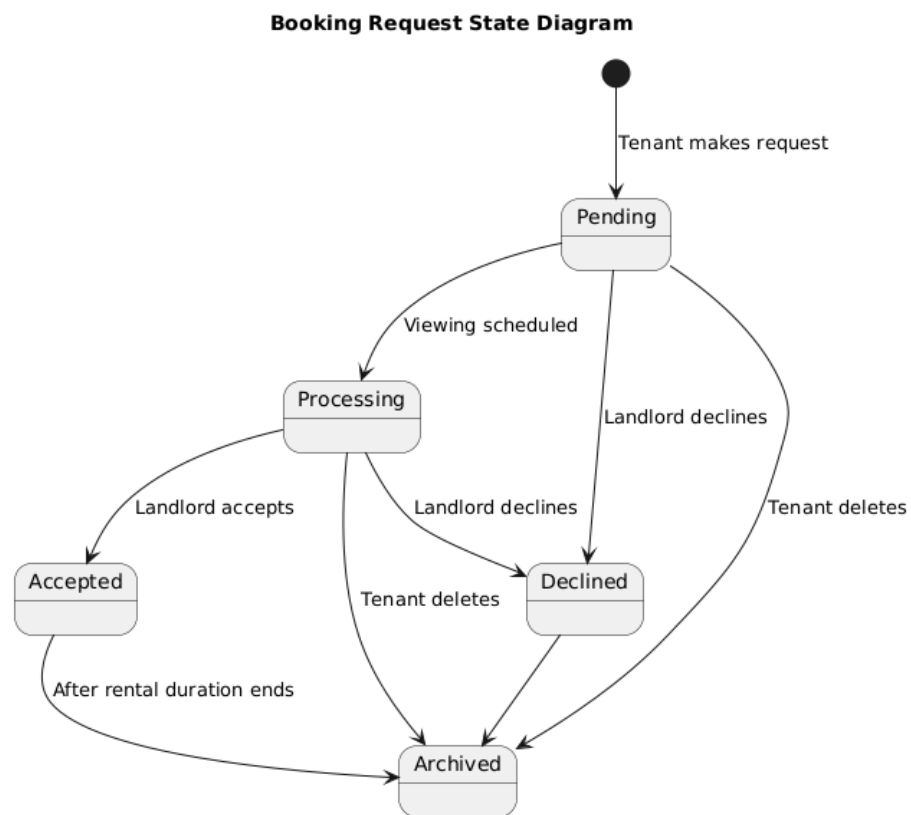


3.3.4. State Chart Diagram

3.3.4.1 Listing State



3.3.4.2 Booking Request



3.4. System Architecture

3.4.1. Sub-System

3.4.1.1. User Management Sub-System

Handle user registration, login, and profile management.

Services:

- User registration with field validation and strong password enforcement.
- Email duplication checking and email verification.
- Google account integration for login and registration.
- Password reset.
- Profile viewing and editing with format validation and confirmation.
- Logout and session termination.

3.4.1.2. Favorites Management Sub-System

Manages the tenant's favorites listings

Services:

- Adding and removing listings from tenant's favorites.
- View tenant's personal favorite listings.

3.4.1.3. Booking Sub-System

Allow tenants to send a booking request

Services:

- Send booking requests with move-in date, duration, and tenant details
- Use of notification sub-system to notify landlords for new booking requests
- Allow users to view their booking request status
- Cancellation of pending requests with confirmation and notifications
- Allow landlord to manage booking requests

3.4.1.4. Listing Sub-System

Manages all listings and their information on the app.

Services:

- Adding, removing, and updating listings.
- Manages the storing of listing images.
- Manages listings lifecycle and status.
- Display available listings.
- Display listing information.
- Use of Cost-Estimator Sub-System to suggest an appropriate price.

3.4.1.5. Chat and Messaging Sub-System

Messaging Engine, Real-Time Communication, Message Storage.

Services:

- Handles the sending, receiving and routing between users.
- Supports real-time message delivery.
- Stores chat messages in a database.

3.4.1.6. Rating and Reviews Sub-System

Rating Mechanism, User Authentication, Ratings Report.

Services:

- Allows users to rate listings (that they rented).
- Allows landlords to rate users (who rented their listings).
- Allows users to view the reviews by date or rating.
- Display average ratings

3.4.1.7. Notification Sub-System

Event Detection, Notification generator and sender

Services:

- Monitoring activities in the application that triggers a notification event
- When the event is detected, the generator creates a notification object for that event.
- Once the object is ready, it will be sent to the user.

3.4.1.8. Cost-Estimator Sub-System

Accept user input, cost calculation, output.

Services:

- Accept Parameters from the landlord (e.g., number of rooms, size).
- Calculate cost using Linear Regression algorithm
- Provide estimated cost of the listing

3.4.1.9. Search-Filtering Sub-System

Handles the filtering and ranking of listings upon user request.

Services:

- Retrieves the appropriate listings based on the user-entered filters.
- Ranks the listings based on match with the user input, price, reviews, and location.

3.4.2. Software Architecture

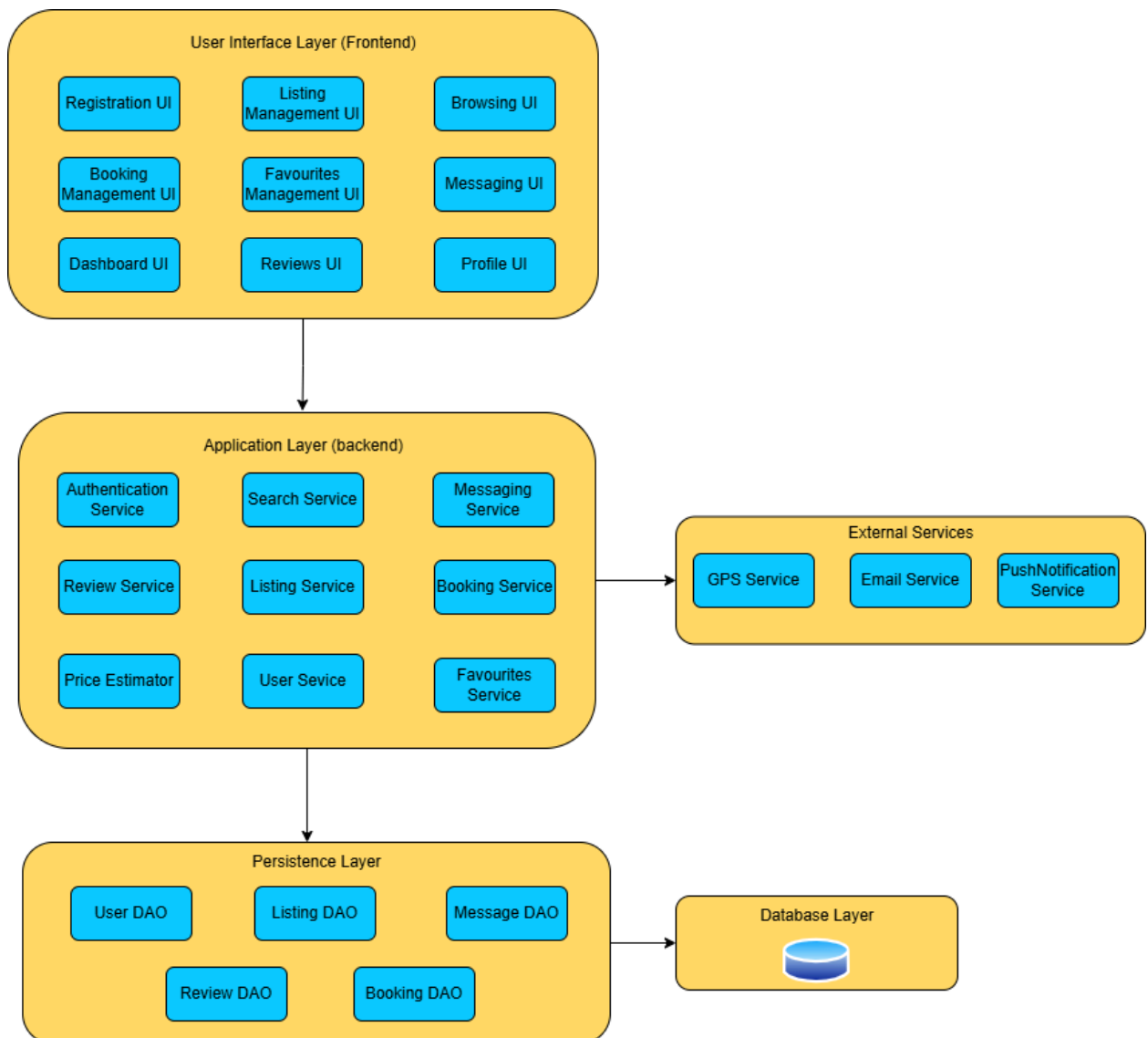


Figure 3-3: Software Architecture

3.4.3. Deployment Diagram

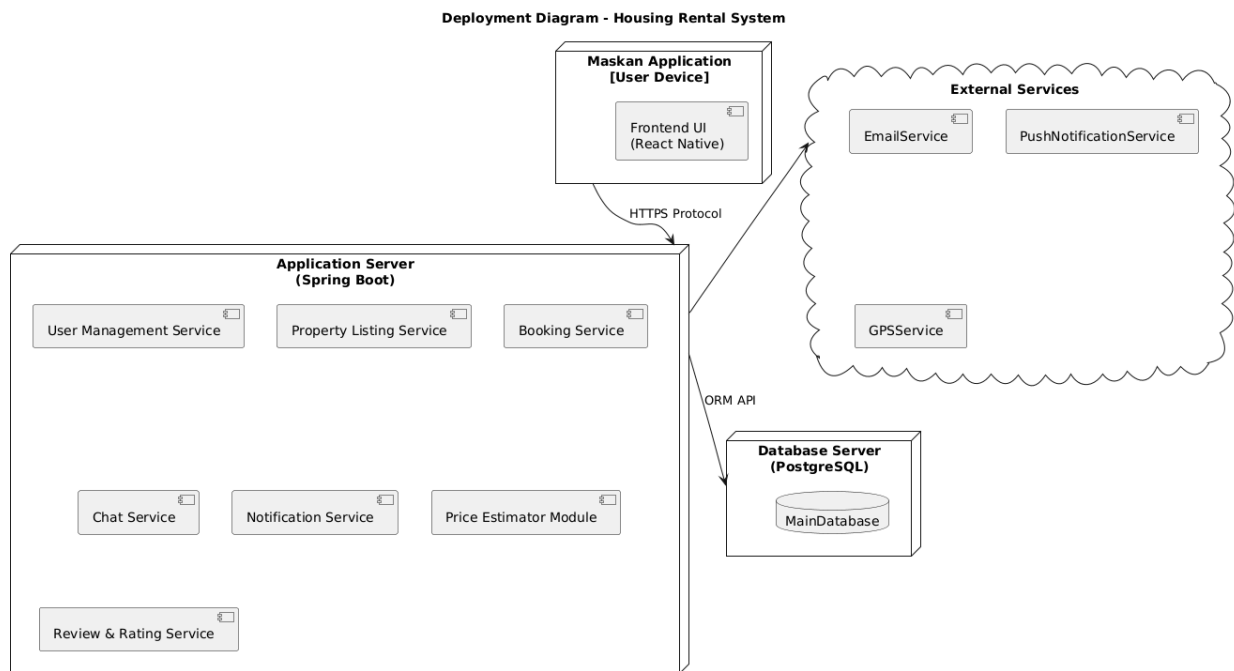
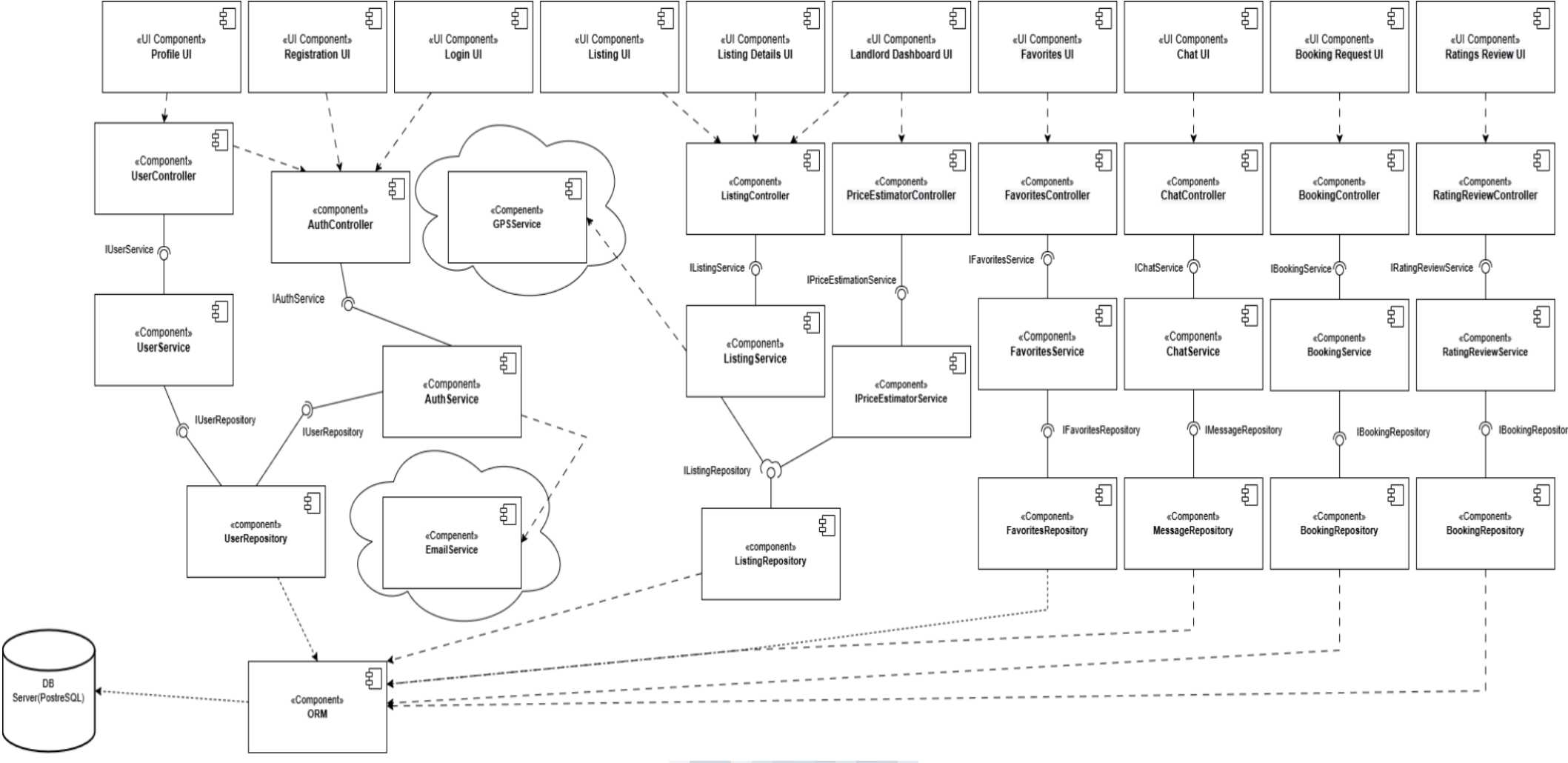


Figure 3-4: Deployment Diagram

3.4.4. Component Diagram



3.5. Data Management and ERD

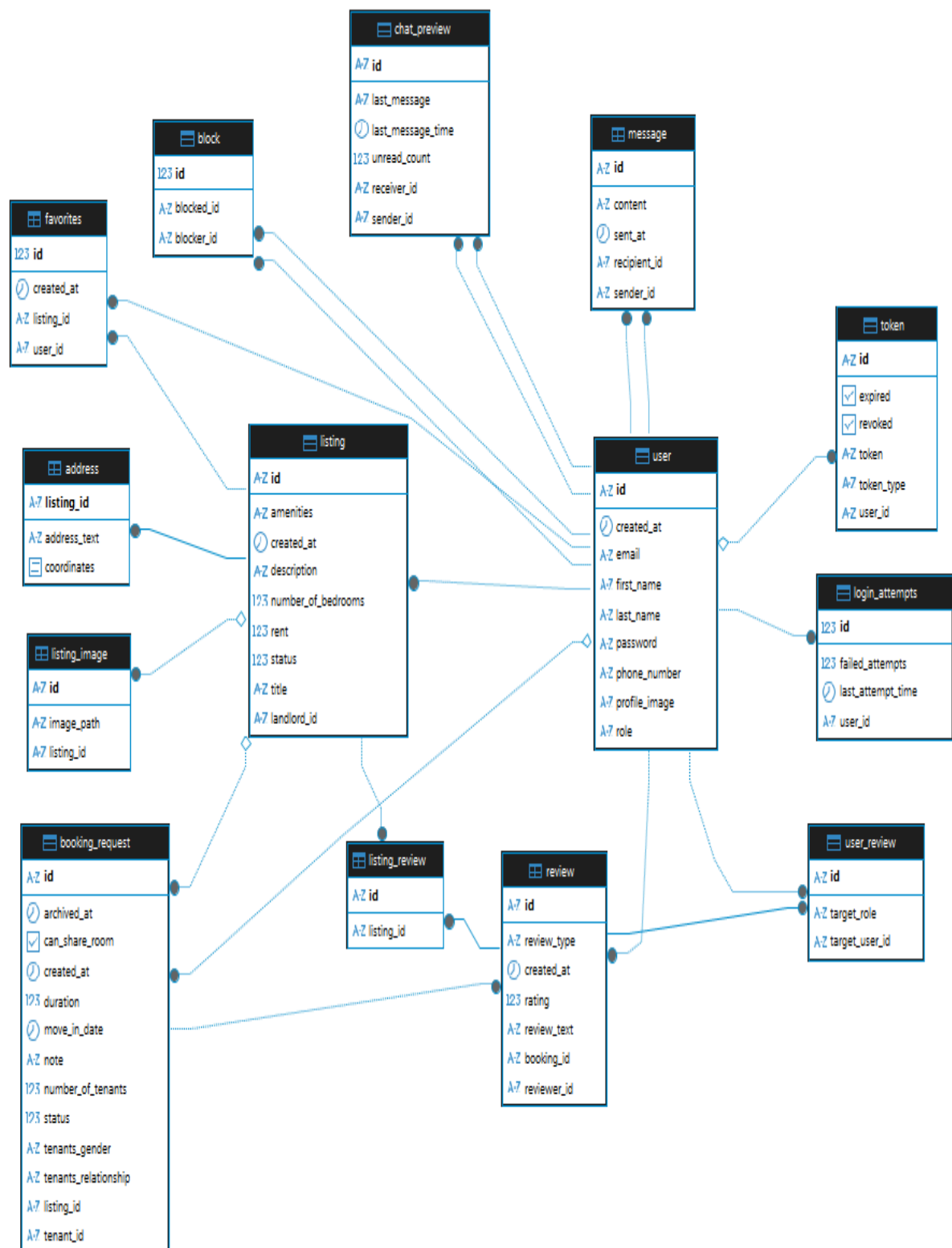


Figure 3-6: ER Diagram

4. Implementation

4.1. Development Environment and Tools

This chapter is about the implementation of the Maskan software system, following an iterative and release-based development approach. The system was developed incrementally through multiple releases, each delivering a complete and functional subset of features. Continuous feedback from the supervisor guided improvements throughout the development lifecycle.

Development and collaboration were supported using:

- **Github**: for version control and source code management.
- **Github Projects**: for task tracking, issue management, and release planning.
- **REST APIs**: for frontend-backend communication.
- **GPS and Google Map service**: for geo-based functionalities.
- **Google push notification service**: for pushing app notifications.

4.2. Release-Based Implementation

The application development was divided into three major releases, each containing a complete functional stage of the features.

1. **Release 1: Core System Setup and Basic Functionality**

The goal of this release was to establish the functional infrastructure and deliver essential system functionality to prepare for future work. **Main Features:**

- i. Project initialization for both frontend (React Native) and backend (Spring Boot)
- ii. PostgreSQL database connection and configuration
- iii. User authentication and registration
- iv. Email Verification and login security
- v. Role-based frontend structure
- vi. CRUD APIs for landlord listings
- vii. Basic login, registration, home page, and listing creation screens

2. **Release 2: Main Business Logic and Use Interactions**

This release focused on implementing the core rental marketplace functionality and user interactions.

a. **Main Features:**

- i. Listing browsing, search, and dynamic filtering
- ii. Geo-based listing discovery
- iii. Booking request creation and management
- iv. Landlord dashboards and tenant dashboards
- v. Favorites list (whishlist) functionality
- vi. Verified review and rating system
- vii. Messaging system between tenants and landlords
- viii. Notification system

- ix. Profile management
 - b. **Dependencies**
 - i. Booking requests depend on listing availability and user authentication.
 - ii. Reviews depend on completed booking interactions
 - iii. Messaging depends on booking relationships between users
3. **Release 3: Advanced Features, Security, and Optimization**
The final release focused on enhancing system reliability and user experience. **Main Features:**
- a. Rental price estimation for landlords
 - b. Listing recommendation system for tenants
 - c. Blocking and block list management
 - d. Prevent blocked users from booking listings.
 - e. Automatic rejection of overlapping booking requests when landlord acceptance.
 - f. Cron jobs for booking and listings status management
 - g. Listing view count for landlords
 - h. UI reinforcement and refactoring
 - i. Multi-language support
 - j. Bug fixes and stability improvements

4.3. Recommendation System Implementation

Our recommendation process is based on a set of interaction attributes that reflect both explicit and implicit user preferences. These attributes include booking request outcomes, where accepted, pending, and declined requests are assigned different weights to represent different levels of user intent, listing views, which capture implicit interest through repeated exposure, and listing reviews, where numerical ratings provide explicit feedback on user satisfaction. All attributes are aggregated into a unified interaction score for each (user, listing) pair.

The aggregated interaction data is transformed into an in-memory recommendation model using **Apache Mahout**. Since Mahout operates on numeric identifiers, a mapping layer is used to convert application-level user and listing identifiers into long values. User similarity is computed using Pearson correlation, and recommendations are generated by identifying the nearest users with similar interaction patterns.

The recommendation model is initialized at application startup and periodically rebuilt to incorporate new interaction data. This design ensures up-to-date recommendations while minimizing runtime performance overhead.

4.4. Price Estimation Implementation

The rental price estimation feature is implemented using a linear regression model trained on historical rental and sale listings collected from the Palestinian real estate market and found on the kaggle platform. The dataset includes key property attributes such as property type, location (city and neighborhood), area size, listing purpose (rent or sale), and currency-adjusted price values.

Data preprocessing involved cleaning inconsistent entries, handling missing values, normalizing prices across different currencies, and encoding categorical features into numerical representations suitable for regression analysis. Since the dataset contains both sale and rental listings, sale entries were removed during the cleaning phase. The trained model captures the relationship between property characteristics and rental price.

The price estimator is integrated into the backend and is invoked by the landlord during listing creation. The system returns an estimated price as a recommendation to assist landlords in setting competitive and realistic prices. The model can be periodically retrained using updated listing data to reflect current market trends while maintaining low computational overhead.

Pseudo code for data cleaning and training the model:

Algorithm TrainPriceEstimationModel(dataset):

```
    cleanedData ← removeInvalidEntries(dataset)
    cleanedData ← removeDuplicateListings(cleanedData)
    cleanedData ← normalizeTextFields(cleanedData)

    rentalData ← filterListings(cleanedData, purpose = "rent")

    rentalData ← parseNumericFields(rentalData, fields = {price,
area})
    rentalData ← normalizeCurrency(rentalData)
    rentalData ← applyRealisticRanges(rentalData)

    encodedData ← encodeCategoricalFeatures(rentalData)

    model ← trainLinearRegression(
        features = propertyAttributes,
        target = rentalPrice
    )

    return model
```

Pseudo code for predicting a listing's price:

Algorithm EstimateRentalPrice(listingData):

```
    encodedListing ← encodeCategoricalFeatures(listingData)
    normalizedListing ← normalizeFeatures(encodedListing)

    estimatedPrice ← regressionModel.predict(normalizedListing)

    return estimatedPrice
```

5. Evaluation

5.1. Add Listing

TEST DATE

Testing Adding a Listing with valid data – Positive test	21/1/2026
--	-----------

TEST SCENARIO

TEST DESIGNED BY

Adding a Listing	Owais Malash
------------------	--------------

TEST DESCRIPTION

PRE-Conditions

Adding a listing with the following data: - Title: “ New Listing” - Description: “ New Listing - Good View” - Area: 200 - Type: House - Facilities: “Furnished, BBQ Area” - Price: 2200 - Photos: Selected Photos. - Number of Rooms: 4 - Location: Selected Coordinates.	Prerequisites: - User Must log in - User type should be landlord
--	--

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click Dashboard	Landlord should be redirected to the Dashboard screen	Correct	Pass	
2	Click Add Listing Option	Landlord should be redirected to the Add Listing screen	Correct	Pass	
3	Fill The data	None	-	-	
4	Click Add Listing Button	Listing should be added to the system and success message should appear	Correct	Pass	
5	Validation Point	Listing is added to the system	Correct	Pass	

5.2. Price Estimator

Testing Price Estimator – positive test	18/1/2026
---	-----------

TEST SCENARIO

TEST DESIGNED BY

Price Estimator	Owais Malash
-----------------	--------------

TEST DESCRIPTION

PRE-Conditions

Testing Price Estimator with the following data: • Area: 200 • Number of Rooms: 4 • Facilities: “Furnished, BBQ Area”	Prerequisites: • User Must log in • User type should be landlord
--	--

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click Dashboard	Landlord should be redirected to the Dashboard screen	Correct	Pass	
2	Click Add Listing Option	Landlord should be redirected to the Add Listing screen	Correct	Pass	
3	Fill The data	None	-	-	
4	Click Estimate Price Button	An Estimated Price should be appeared near the price field	Correct	Pass	
5	Validation Point	An estimated price is shown next to the price field	Correct	Pass	

5.3. Edit Listing

TEST DATE

Testing Edit Listing With Invalid Data – Negative test	21/1/2026
--	-----------

TEST SCENARIO

TEST DESIGNED BY

Edit Listing	Owais Malash
--------------	--------------

TEST DESCRIPTION

PRE-Conditions

Testing Edit Listing with invalid Data:: Remove All images.	Prerequisites: - User Must log in - User type should be landlord - A listing should be in the system for the logged in user
--	--

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click Dashboard	Landlord should be redirected to the Dashboard screen	Correct	Pass	
2	Click Listings Icon	Landlord should be redirected to the Listings screen	Correct	Pass	
3	Click a Listing	Landlord should be redirected to a screen showing the listing details.	Correct	Pass	
4	Scroll Down and Click Edit Listing	Landlord should be redirected to the Edit Listing screen and the listing data should be filled.	Correct	Pass	
5	Remove All Images	Images Should be removed, Warning must be appeared indicating that user must at least an image	Correct	Pass	
6	Click Update Listing	The Listing shouldn't be updated and alert must be shown indicating the problem	Correct	Pass	
7	Validation Point	Listing data isn't updated on the system and the user is informed	Correct	Pass	

5.4. Registration

TEST DATE

Registration new user – Positive test	19/1/2026
---------------------------------------	-----------

TEST SCENARIO

TEST DESIGNED BY

Registration new user	Owais Malash
-----------------------	--------------

TEST DESCRIPTION

PRE-Conditions

Testing Registration with this data:: - Unique email : unique@gmail.com - First Name: New - Last Name: User - Phone number: validPhoneNumber. - Password: New_user123 - User Type: Tenant	Prerequisites: None
---	--

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click Get Started	User Should be redirected to the Login/Registration Page	Correct	Pass	
2	Click Create an Account	User should be redirected to the enter email screen	Correct	Pass	
3	Enter Unique And Valid email	none			
4	Click get Code	User should get code on the entered email	Correct	Pass	
5	Enter The code	User should be redirected to the Registration page	Correct	Pass	
6	Fill the data	none			
7	Click Submit	User should be registered and redirected to the Home Page	Correct	Pass	
8	Validation Point	User is registered on the system and redirected to Home Page	Correct	Pass	

5.5. Send Booking Request

TEST DATE

Send Booking Request – Positive test	20/1/2026
--------------------------------------	-----------

TEST SCENARIO

TEST DESIGNED BY

Send Booking Request	Baha Al-Khateeb
----------------------	-----------------

TEST DESCRIPTION

PRE-Conditions

Sending Booking Request with this data: - Note: “Testing Booking Request” - Can Share Room: No - Number Of Tenants: 3 - Tenants Gender: Mix - Tenant Relationship: Family - Move-in-Date: 22/1/2026 - Duration: 4 months.	Prerequisites: - User Must log in - User type should be Tenant - Available listing should be in the system.
--	--

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Browse a listing or Click the available in the Home Page	Tenant should be redirected to the Listing details screen	Correct	Pass	
2	Click Send Request button	Tenant should be redirected to the Send Request Screen	Correct	Pass	
3	Fill data	none			
4	Click Submit Request	User Should be alerted that the booking request was submitted successfully and redirected again to the listing details screen	Correct	Pass	
5	Validation Point	Booking request is saved to the system and the user is informed of the success	Correct	Pass	

5.6. Send Duplicate Booking Request

TEST DATE

Send Duplicate Booking Request – Negative test	18/1/2026
--	-----------

TEST SCENARIO

TEST DESIGNED BY

Send Duplicate Booking Request	Baha Al-Khateeb
--------------------------------	-----------------

TEST DESCRIPTION

PRE-Conditions

Sending Booking Request with this data: - Note: "Testing Booking Request" - Can Share Room: No - Number Of Tenants: 3 - Tenants Gender: Mix - Tenant Relationship: Family - Move-in-Date: 22/1/2026 - Duration: 4 months.	Prerequisites: - User Must log in - User type should be Tenant - Available listing should be in the system.
--	--

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Browse a listing or Click the available in the Home Page	Tenant should be redirected to the Listing details screen	Correct	Pass	
2	Click Send Request button	Tenant should be redirected to the Send Request Screen	Correct	Pass	
3	Fill data	none			
4	Click Submit Request	Tenant Should be alerted that the booking request was submitted successfully and redirected again to the listing details screen	Correct	Pass	
5	Click Send Request button	Tenant should be blocked from sending and get a message that he already sent a request.	Correct	Pass	
6	Validation Point	The Tenant is informed of the duplicated request	Correct	Pass	

5.7. Browse Listings By Price Filters

TEST DATE

Browse Listings By Price Filters – Positive test	21/1/2026
--	-----------

TEST SCENARIO

TEST DESIGNED BY

Browse listings by price filters	Baha Al-Khateeb
----------------------------------	-----------------

TEST DESCRIPTION

PRE-Conditions

Browsing listings by these filter: Price range: 2000 to 3000	Prerequisites: - User Must log in - Available listings should be in the system and its price should be in the 2000 to 3000 range.
---	---

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click Search Icon	User should be redirected to the search screen	Correct	Pass	
2	Click on the filter icon in the search bar	Filters should appear	Correct	Pass	
3	Fill the rent fields 2000 and 3000	none	-	-	
4	Click Apply filters	User is redirected to the search results page where only listing with a price in the range 2000 to 3000 should appear	Correct	Pass	
6	Validation Point	Listing in the price range 2000 to 3000 appear to the user	Correct	Pass	

5.8. Initiate A Chat with a Tenant

TEST DATE

Initiate a chat with a tenant – Positive test	19/1/2026
---	-----------

TEST SCENARIO

TEST DESIGNED BY

Initiate a chat with a tenant	Omar Batran
-------------------------------	-------------

TEST DESCRIPTION

PRE-Conditions

Initiate a chat with a tenant that has sent a booking request.	Prerequisites: • User Must log in • User type must be landlord • A tenant must have sent a booking request to the logged in landlord
--	---

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click Dashboard Icon	Landlord should be redirected to the dashboard screen	Correct	Pass	
2	Click on the Requests Icon	Landlord should be redirected to the requests screen	Correct	Pass	
3	Click on the pending booking request	Landlord should be redirected to the booking request details	Correct	Pass	
4	Click on the chat button	Landlord should be redirected to the chat screen and can send a message to the tenant	Correct	Pass	
6	Validation Point	Landlord is directed to the chat screen	Correct	Pass	

5.9. Block A User

TEST DATE

Block A User – Positive test	21/1/2026
------------------------------	-----------

TEST SCENARIO

TEST DESIGNED BY

Block a tenant by a landlord	Omar Batran
------------------------------	-------------

TEST DESCRIPTION

PRE-Conditions

Blocking a tenant by a landlord	Prerequisites: • User Must log in • User Type must be landlord • A chat with a tenant must exist
---------------------------------	---

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click on the chats icon	Landlord should be redirected to the available chats	Correct	Pass	
2	Click on an available chat	Landlord should be redirected to the Chat screen	Correct	Pass	
3	Click on the user name or photo	Landlord should be redirected to the user details screen	Correct	Pass	
4	Click Block button	User should be blocked	Correct	Pass	
6	Validation Point	User should be blocked	Correct	Pass	

5.10. Accept a Booking Request

TEST DATE

Testing accepting a booking request – Positive test	21/1/2026
---	-----------

TEST SCENARIO

TEST DESIGNED BY

Accepting a booking request as a landlord	Omar Batran
---	-------------

TEST DESCRIPTION

PRE-Conditions

Accepting a received booking request as a landlord	Prerequisites: • User Must log in • User Type must be landlord
--	--

STEP ID	STEP DESCRIPTION	EXPECTED RESULTS	ACTUAL RESULTS	PASS / FAIL	ADDITIONAL NOTES
1	Click Dashboard	Landlord should be redirected to the Dashboard screen	Correct	Pass	
2	Click on Requests Button	Landlord should be redirected to the requests screen	Correct	Pass	
3	Click on a booking request	Landlord should be redirected to the booking request details screen	Correct	Pass	
4	Click Accept button	The booking request should be accepted, saved to the system, and status on the screen should be updated	Correct	Pass	
6	Validation Point	The booking request is accepted	Correct	Pass	

6. Conclusion and Future works

This chapter presents a comprehensive conclusion of the Maskan graduation project, summarizing the implemented system, reflecting on the achieved objectives, discussing current limitations, and outlining potential directions for future development.

1. Project Summary

The Maskan app was designed and implemented to address the challenges faced by apartment tenants and landlords in the rental market, such as scattered listings, inefficient search processes, and limited communication tools. Throughout this graduation project, the system evolved from a conceptual design into a fully functional mobile application supported by a robust backend.

The final system successfully connects landlords and tenants through a unified platform. Landlords are able to list and manage their properties, receive automated rent price suggestions, handle booking requests, and communicate directly with potential tenants. On the other hand, tenants can browse and search apartments using advanced filters, save favorite listings, send booking requests, and communicate with landlords through real-time chat.

Additionally, the system integrates intelligent components such as a dynamic recommendation system based on user behavior, and a price estimation model to assist landlords in setting competitive rental prices. By utilizing modern technologies such as React Native, Spring Boot, PostgreSQL, and Docker, Maskan achieves cross-platform compatibility, scalability, reliable performance, and easy deployment.

Overall, the project meets its main objectives and delivers a practical solution that improves efficiency, transparency, and user experience in the apartment rental process.

2. Limitations

Despite the successful implementation of the core system, several limitations remain:

1. The system does not currently include a dedicated admin dashboard. As a result, tasks such as global user management, content moderation, system monitoring, and dispute handling are not supported.
2. The price estimation algorithms rely on limited data (about 300 rows), which may affect accuracy of the suggested prices.
3. Due to time constraints, extensive usability testing and performance optimization under heavy load were not fully conducted.

These limitations are primarily the result of project scope and time limitations.

3. Future Work

Several improvements and extensions can be considered for future versions of Maskan:

1. Implementing a full admin dashboard to manage users, listings, reviews, reports, and system configurations.

2. Enhancing the price estimator by incorporating more advanced machine learning models and larger datasets.
3. Improving scalability and performance through caching mechanisms and load balancing.
4. Adding advanced security features such as enhanced fraud detection and automated content moderation.
5. Expanding platform support by introducing a web-based version alongside the mobile application.
6. Conducting extensive user testing to further refine usability and user experience.

In conclusion, Maskan provides a strong foundation for a smart apartment rental platform. With further enhancements and continued development, it has the potential to evolve into a comprehensive and scalable solution for the housing rental market.

References

- [1] <https://www.rentvision.com/blog/apartment-website-problems-and-how-to-fix-them>
- [2] <https://www.irjet.net/archives/V9/i5/IRJET-V9I565.pdf>
- [3] James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). An Introduction to Statistical Learning.
<https://www.statlearning.com/>
- [4] Montgomery, D. C., Peck, E. A., & Vining, G. G. (2021). Introduction to Linear Regression Analysis.
- [5] Draper, N. R., & Smith, H. (1998). Applied Regression Analysis.
- [6] StatQuest - Linear Regression Explained https://www.youtube.com/watch?v=nk2CQITm_eo
- [7] <https://codewave.com/insights/ecommerce-recommendation-algorithms/>
- [8] <https://www.turing.com/kb/collaborative-filtering-in-recommender-system>
- [9] <https://insights.daffodilsw.com/blog/machine-learning-algorithms-for-recommendation-engines>
- [10] <https://www.atlantis-pess.com/article/125985505.pdf>
- [11] https://www.researchgate.net/publication/341901838_Smart_Rent_Portal_using_Recommendation_System_Visualized_by_Augmented_Reality
- [12] <https://www.geeksforgeeks.org/collaborative-filtering-ml/>
- [13] https://www.researchgate.net/publication/263377228_Hybrid_Recommender_Systems_Survey_and_Experiments
- [14] https://didawiki.di.unipi.it/lib/exe/fetch.php/magistraleinformatica/ir/ir13/1_-_learning_to_rank.pdf
- [15] https://arxiv.org/pdf/2212.12343_page_6